



BACHELOR'S THESIS

By:

Student name: HOANG Viet Thanh BI12-408

Major: Information and Communication Technology

Title:

Investigation of IoT-Based Monitoring and Alert System For Machine Room Safety Using Raspberry Pi

Dr. PHAM Ngoc Minh

Thesis Supervisor

IOIT Institute of Information Technology

MSc. LE Nhu Chu Hiep

Co-Thesis Supervisor

USTH University of Science and Technology of Hanoi

Hanoi, 2025

To whom it may concern,

I, Pham Ngoc Minh, certify that the thesis/ internship report of Mr/Ms. Hoang Viet Thanh is qualified to be presented in the Internship for July 2024-2025.

Hanoi, 04 July 2025

Supervisor's signature



Pham Ngoc Minh

ABSTRACT

In recent years, fire accidents – particularly in enclosed and high-risk environments have become increasingly common, leading to severe damages and operational disruptions. The development of IoT fire monitoring and alarm systems has received significant attention, but challenges remain in terms of cost, real-time response, and infrastructure complexity. To solve these limitations and enable accurate, complete monitoring, this thesis investigates the use of lightweight wireless communication protocols that do not require existing infrastructure. Sensor data is securely encrypted and transmitted from the nodes to a central processing unit for real-time analysis, thereby increasing system stability and accuracy.

This thesis proposes a real-time IoT-based fire monitoring and alert platform that utilizes multiple distributed sensor nodes communicating via low-power wireless channels. Data is visualized through a web-based interface to ensure user-friendly interaction. In addition, a Fuzzy Logic algorithm is used to evaluate the potential for a fire based on temperature, humidity, and other sensor data. The technology automatically sends out warnings over email and sounds alarms, providing quick and efficient reactions to any risk of fire.

AUTHORSHIP

I have completed my bachelor's thesis, **“Investigation of IoT-Based Monitoring and Alert System for Machine Room Safety Using Raspberry Pi,”** as a result of my diligent study and research under the guidance of my academic supervisors. In the reference part of this thesis, I have completely and correctly referenced all sources used during the research process.

I hereby confirm that the facts and results contained in this thesis are true and correct. I assume full responsibility for any errors or instances of academic misconduct (if any). I will accept any punishments given by the University of Science and Technology of Hanoi (USTH) to comply with its statutes.

Hanoi, July 4, 2025

Author



Hoàng Việt Thanh

ACKNOWLEDGEMENT

I want to express my heartfelt gratitude to all the professors and colleagues who have supported and guided me throughout my academic journey. I would also like to extend my sincere thanks to the University of Science and Technology of Hanoi (USTH) for supporting me from the early days of my university studies to the completion of this graduation thesis.

First and foremost, I would like to express my sincere gratitude to my thesis supervisor, Mr. PHAM Ngoc Minh from the IOIT, for the opportunity to intern in his embedded lab. His guidance and suggestions greatly shaped the direction of this project. His mentorship broadened my major to include not only software but also hardware and embedded systems. I also thank Mr. NGO Duy Tan from VNSC for his creative input and inspiration throughout the thesis development. I sincerely thank my co-supervisor, Mr. LE Nhu Chu Hiep from USTH, for his technical guidance and support in system deployment and for helping refine my final report.

Furthermore, I would like to thank my friends, Mr. HOANG Van Son, Mr. NGUYEN Trung Kien, NGUYEN Nhat Anh, DOAN Quang and HO Cong Thanh, for their constant support and companionship throughout this journey. Lastly, sincerely thank Ms. NGUYEN Minh Huong for her understanding and compassion.

It is impossible not to mention my parents and sister. I wish to express my deepest gratitude to my family, whose unconditional love and constant encouragement have been the foundation of my enduring strength. I am eternally thankful for their unwavering support throughout this journey.

“There’s nothing more rewarding than working with someone who is smarter than you are, so go find people who make you feel stupid in comparison.” - Russell Merrick.

Finally, having the opportunity to work with, receive guidance and support from, and learn from supervisors, brothers, and friends who are more knowledgeable than I is a valuable experience that has helped me grow and complete this thesis. Once again, I want to extend my sincere thanks to everyone.

Contents

ABSTRACT	ii
AUTHORSHIP	iii
ACKNOWLEDGEMENT	iv
Contents	v
List of Abbreviations	vi
List of Figures	viii
List of Tables	x
Introduction	xi
Chapter 1. Overview of the Monitoring and Alert System.....	1
1.1. Introduction to the Internet of Things (IoT) and Its Applications for Fire Monitoring Alert Systems	1
1.2. Current Situation and Limitations of the Monitoring Fire Alert System	2
1.3. Related Works.....	3
1.4. Conclusions.....	5
Chapter 2. System Design	6
2.1. Hardware System Architecture.....	7
2.1.1. Sensor Node Architecture	8
2.1.2. Central Processing Unit Architecture	11
2.2. Software & Firmware System Architecture	12
2.2.1. Data Processing at the Sensor Node (The Transmitter).....	13
2.2.2. Data Processing at Central Processing Unit (The Receiver)	17
2.2.3. Software Development at Central Processing Unit	20
2.2.4. System Alert.....	26
2.3. Conclusions.....	30
Chapter 3. Results and Evaluations.....	31
3.1. Implementation	31
3.2. Reliability and Error Rates	39
3.3. Evaluations and Challenges	40
3.4. Conclusion	41
Conclusions and Perspective	42
References.....	43

List of Abbreviations

Abbreviation	Definition
ADC	Analog-to-Digital Converter
AES-256	Advanced Encryption Standard with 256-bit key
AI	Artificial Intelligence
App	Application
ARM	Advanced RISC Machines
ASUS	ASUSTeK Computer Inc
CMake	Cross-Platform Make
COMP ID	Component ID
CPU	Central Processing Unit
CRC	Checksum
dBi	Decibel isotropic (gain)
DHT11	Digital Humidity and Temperature Sensor
ESP32	Espressif Systems Platform (32-bit)
GB	Gigabyte
GPIO	General Purpose Input/Output
GPIOA	General Purpose Input/Output (Port A)
GSM	Global System for Mobile Communications
HC-12	HopeRF Communication 12 (Series)
IEEE	Institute of Electrical and Electronics Engineers
IOIT	Institute of Information Technology
IoT	Internet of Things
KY-026	Keyes (Company) Module #026 – Flame Sensor Module
LCD	Liquid Crystal Display
LDPC 1/2	Low-Density Parity-Check Code (Rate 1/2)
LEN	Length
LFSR	Linear Feedback Shift Register
LoRa	Long Range
MAVLink	Micro Air Vehicle Link
MHz	Megahertz
microSD	Micro Secure Digital
MSG ID	Message ID
PCB	Printed Circuit Board
PIR	Passive Infrared Sensor
PKCS7	Public-Key Cryptography Standards #7
RAM	Random Access Memory

RF	Radio Frequency
Rx	Receive
SBC	Single-Board Computer
SEQ	Sequence
SMA	SubMiniature version A
SMS	Short Message Service
STM32	STMicroelectronics 32-bit Microcontroller
STX	Start of frame
SWR	Standing Wave Ratio
SYS ID	System ID
TB	Terabyte
Tx	Transmit
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
USTH	University of Science and Technology of Hanoi
VNSC	Vietnam National Space Center
VPN	Virtual Private Network
W	Watt
Wi-Fi	Wireless Fidelity
XOR	Exclusive OR
YOLOv5	You Only Look Once (version 5)

List of Figures

Figure 1. An IoT system with interconnected smart devices [1].....	2
Figure 2: System Architecture Diagram.....	6
Figure 3: Sensor Node Architecture.	7
Figure 4: Central Processing Unit Architecture.	8
Figure 5: KY-026 – Flame sensor [11].....	8
Figure 6: DHT11 - Temperature & Humidity Sensor [12].	9
Figure 7: STM32F103C8T6 Microcontroller [13].....	9
Figure 8: HC-12 – Wireless Communication Module [14].....	10
Figure 9: 433 MHz RF Antenna [15].	10
Figure 10: Sensor Node Circuit Diagram.....	11
Figure 11: Raspberry Pi 5 – Single board Computer[16].....	12
Figure 12: ASUS ZenScreen Touchscreen [18].	12
Figure 13: Data Processing at the Sensor Node.	13
Figure 14: Data protection sequence in wireless transmission.....	14
Figure 15: MAVlink v1 Frame [20].	15
Figure 16: Data Processing at the Central Processing Unit.....	17
Figure 17: Flow Decoding message at the CPU.....	18
Figure 18: Software Architecture diagram.	20
Figure 19: Use case diagram.	23
Figure 20: Export Logs Sequence Diagram.	24
Figure 21: Threshold Setting Sequence Diagram.....	25
Figure 22: Deactivate Alert Sequence Diagram.....	25
Figure 23: Alert Sequence Diagram.	26
Figure 24: Fuzzy Membership Functions for Water State (Cold, Warm, Hot).....	27
Figure 25: Membership of Temperature.....	28
Figure 26: Membership of Humidity.....	28
Figure 27: 3D Sensor Node Circuit Diagram.	31
Figure 28: PCB for Sensor Node.....	32
Figure 29: Edge Processor with various sensors and a wireless module.	32

Figure 30: Debug STM32 IDE Temp, Humi, Fire state value.	32
Figure 31: Sensor Node.	33
Figure 32: Central Processing Unit.	34
Figure 33: Software Interface – Login.	34
Figure 34: Software Interface.	35
Figure 35: Software Interface – Dashboard.....	35
Figure 36: Software Interface – Threshold Settings.....	36
Figure 37: Software Interface – Dashboard (Fire).	36
Figure 38: Software Interface – Fire Events.....	37
Figure 39: Software Interface – Warning Notification.	37
Figure 40: Software Interface – List and Export Data Day.....	38
Figure 41: Software Interface – List and Export Fire Event.	38
Figure 42: The IoT-based monitoring and alert system.	39

List of Tables

Table 1: Functional requirement.22

Table 2: Rule of Temperature, Humidity, Fire status.....29

Introduction

In recent years, the risk of fire in enclosed environments, such as machine rooms, has become a growing concern, for example, due to the high density of heat-generating equipment and the presence of flammable materials. Traditional fire alarm systems often lack real-time monitoring, require constant human supervision, and have limited remote accessibility and intelligent response mechanisms. Therefore, quick detection and rapid response are crucial to minimizing damage and ensuring safety.

With the advancement of the Internet of Things (IoT), smart fire monitoring systems have emerged as a promising solution to overcome these limitations. IoT enables the integration of multiple environmental sensors, real-time data transmission, and remote control, allowing for more accurate and efficient fire detection. Moreover, the use of hardware platforms such as the Raspberry Pi makes it easier and more cost-effective to develop custom, scalable systems.

This thesis focuses on investigating the design and implementation of an IoT-based fire monitoring and alert system tailored explicitly for machine room environments. The system utilizes multiple sensors, including temperature, humidity, and flame detection modules, which are integrated into the node connected to a Raspberry Pi, serving as the Central Processing Unit (CPU). This system utilizes wireless communication protocols for data transmission and visualizes sensor data through a web-based interface, allowing for remote monitoring and data tracking. Additionally, Fuzzy Logic and threshold-based alert mechanisms are implemented to evaluate fire risk levels, triggering notifications through email and activating sound alarms to ensure timely alerts and responses. Finally, the thesis will evaluate the IoT system's performance, interaction, and accuracy.

Chapter 1. Overview of the Monitoring and Alert System

This chapter provides an overview of the application of Internet of Things (IoT) technology in monitoring and alerting fire detection systems, primarily in the context of increasingly serious fire problems that require critical attention. It begins with an introduction to IoT and the importance of its role in improving monitoring skills and its value in fire situations. In addition, this chapter also focuses on the main objective of the thesis: to explore and employ the full potential of IoT-connected hardware devices in both IoT applications and real-world scenarios. This chapter is organized into three sections. *Section 1.1* provides an overview of the IoT and its applications in fire monitoring systems. *Section 1.2* discusses the current situation and limitations of existing fire monitoring systems. *Section 1.3* presents modern fire detection methods using current technologies, along with their limitations.

1.1. Introduction to the Internet of Things (IoT) and Its Applications for Fire Monitoring Alert Systems

In recent years, fire accidents have become increasingly dangerous, causing critical damage to both human life and property. Traditional fire monitoring systems, which often rely on manual checks or local alerts, are limited in their ability to respond quickly and effectively, especially in complex or remote environments. To handle this issue, we need smart and effective solutions to improve it in time.

The IoT is one exciting technical development that resolves these issues. As its name suggests, IoT refers to a network of smart devices that can communicate and connect with each other, exchanging data. These devices can collect, transmit, and process data in real-time, thereby enabling automation and making personal decisions.

Unlike standalone devices that work independently, a true IoT system (Figure 1) is a scalable and integrated system where multiple devices work together for a common goal. In the context of fire detection, this goal is always monitoring environmental conditions, identifying early signs of fire, and sending real-time alerts, even when no one is present at that location.

By providing features such as remote monitoring, automated notifications, and log data, IoT-based systems significantly enhance responsiveness and overall safety. Therefore, using IoT for fire monitoring systems is both useful and essential for enhancing early detection and minimizing damage.



Figure 1. An IoT system with interconnected smart devices [1].

1.2. Current Situation and Limitations of the Monitoring Fire Alert System

Fire accidents are now common and can occur in various types of environments, ranging from crowded residential areas to industrial zones, forests, etc,.. In an industrial environment, the use of flammable materials, high electricity consumption, and poor ventilation are common reasons that increase the risk of fire. In a residential area, the complex and overloaded electrical network increases the chance of short circuits and electrical fires. In a forest environment, rising global temperatures make it more flammable, and also human activities, such as camping or purposeful burning of the forest, increase the risk of fire. As mentioned, the fire can occur anywhere, any time, whether in crowded urban areas or isolated natural areas.

Therefore, deploying fire monitoring systems with sensors and data-collecting modules in large areas is expensive and challenging. Traditional systems, which rely on wired connections or require continuous human monitoring, are not realistic, especially in large areas or remote and harsh environments. These systems will face problems such as processing complex data and false fire alerts. This is

really a big challenge for the fire warning system. The solution using IoT with sensor data can help provide more accurate and useful results.

Currently, traditional fire monitoring and alert systems are still widely used and have not been fully replaced by IoT-based systems. Although some improvements have been made, many issues and limitations still exist, such as detection delays, limited coverage range, and high maintenance requirements. As a result, developing a fire detection and monitoring system that uses fire, temperature, and humidity sensors combined with wireless communication technologies is a creative, possible, worthy solution to consider.

1.3. Related Works

The development of fire monitoring and alert systems based on IoT has gained increasing attention in recent years. This section presents several existing works and discusses their limitations, highlighting the gap that this project aims to address.

Currently, IoT systems typically integrate various sensors and microcontrollers, ranging from simple to advanced designs. Moreover, with the growing popularity of artificial intelligence (AI), many modern IoT applications are incorporating machine learning to improve detection accuracy. Although this thesis does not apply AI, it is planned as a direction for future work.

Regarding related works, one example is presented in [2], the authors proposed an IoT-based fire detection and monitoring system using temperature, flame, and smoke sensors, paired with an ESP32 microcontroller. Their system employed a LoRa network to detect and alert about forest and farm fires. Similarly, in [3], the authors developed an Arduino-based home fire alarm system that uses a GSM module and temperature sensor to send SMS alerts, thereby enhancing user safety and protecting property. In another study [4], the researchers designed a smart IoT-based system that also used temperature, flame, and smoke sensors, like in [2], but combined them with an Arduino UNO and a gas sensor. Their system was implemented on the Blynk platform and used GSM communication for alerting. Meanwhile, in [5], an STM32-based wireless fire detection system was introduced,

utilizing multiple sensors and embedded wireless technologies such as Wi-Fi to detect fires in real-time – the system monitors through a mobile phone App and an LCD in the home. A more advanced system is shown in [6], which focuses on the use of robotics in fire monitoring. It integrates STM32 and STC89C51 microcontrollers, along with drones, to create a combined air and ground monitoring system, with communication facilitated through ZigBee. On the other hand, [7] emphasizes the alerting. This system uses an ESP32 with PIR sensors to detect both fire and movement and sends alerts through a Telegram bot to smartphones, including visual and temperature-based alarms.

As AI continues to develop, some recent papers have begun integrating it with IoT for fire detection. For instance, [8] presents a system that combines IoT devices with the YOLOv5 model to enable early and real-time forest fire detection, aiming to reduce false alarms and enhance safety during dry seasons. Building upon this, [9] proposes an improved system that uses IoT and machine learning, integrating various sensor types to enhance detection accuracy and facilitate remote monitoring. Although recent works reflect technological advancements, the core concept of IoT-based fire detection was explored much earlier. Significantly, [10] the paper describes a real-time fire alarm system developed using Raspberry Pi and Arduino Uno. This system included smoke detection, room image capture, and alerting through SMS and a web interface. Moreover, it featured a user confirmation step before notifying firefighters, effectively reducing false alarms. Despite having a basic web interface, the implementation was comprehensive and forward-thinking. Therefore, this paper provides strong motivation to support this thesis.

As stated earlier, while this thesis does not yet include AI integration, it aims to build upon these foundational ideas. Notably, many existing systems use Wi-Fi or GSM for communication. In contrast, this thesis proposes an enhanced monitoring and alert fire system by adopting radio communication as the core of an IoT-based fire detection and alert system. Ultimately, this work is made possible thanks to the pioneering efforts of these earlier researchers, upon whose contributions this thesis continues to build.

1.4. Conclusions

An IoT system is an essential technology with applications in monitoring and fire detection. Achieving real-time monitoring, high accuracy, processing all scenarios, and overcoming hardware limitations creates serious difficulties. The use of IoT brings a promising solution, enabling real-time monitoring, remote access, and automated alert notifications. This project aims to inherit and integrate the ideas from those previous works, while also contributing further improvements to develop a more responsive and reliable fire alert system. It also introduces new innovations to improve the system's flexibility and performance in real-life situations. In Chapter 2 *System Design*, a proposal will be presented, focusing on the integration of hardware and software components, including sensor modules, communication systems, and control logic. All of which will be implemented through software to build the proposed IoT-based fire monitoring solution.

Chapter 2. System Design

The first chapter of this thesis provided an overview of IoT technology and its applications in fire detection and monitoring. It also discussed the current state of fire accidents occurring in many places today, highlighting the need for more efficient solutions. Furthermore, in light of the current state of fire, the thesis proposes a new direction by enhancing existing ideas through the use of radio communication. While many previous systems deploy on Wi-Fi or GSM networks, this thesis focuses on utilizing *low-cost, low-power* wireless communication modules. The main goal is to develop an IoT-based system that monitors, detects fires, and alerts in real-time.

Building on this ideal, *Chapter 2* presents the proposed system architecture, which consists of a system architecture diagram (Figure 2) show what the entire system is like, including hardware and software components.

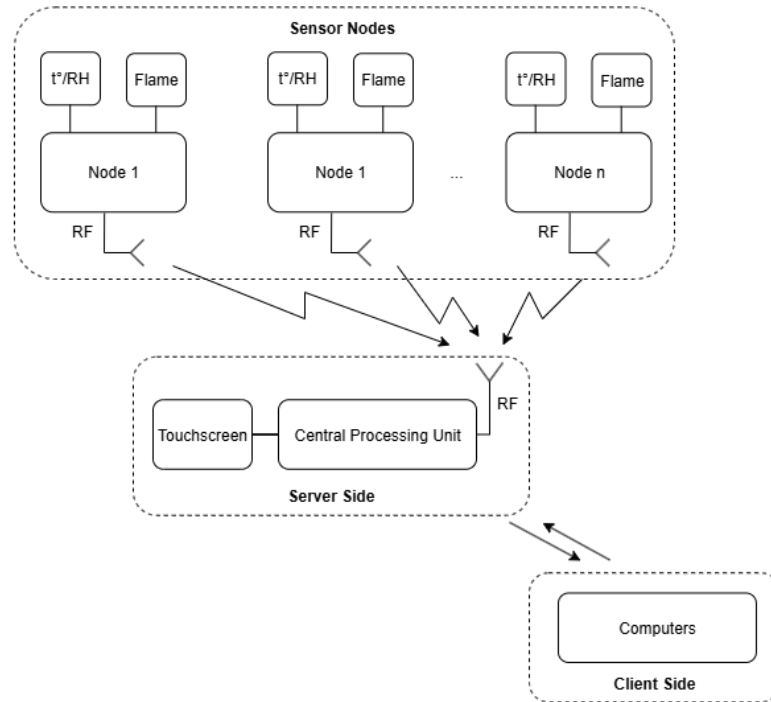


Figure 2: System Architecture Diagram.

The system includes: Raspberry Pi and STM32 microcontrollers, HC-12 modules for communication, and a variety of sensors to collect fire-related data. This chapter has three main sections: *Section 2.1 Hardware System Architecture*: In this section, an overview of the hardware components used in this thesis will be presented, including

sensors, microcontrollers, and the central processing unit. Each component will be explained with the reason why it is used and a summary of its advantages and disadvantages. *Section 2.2: Software & Firmware System Architecture:* This part details how the software & firmware work across the system. It begins with sensor-side firmware, which includes collecting data, encoding, and transmission. On the server side, it explains how data is received, decoded, stored, and visualized through a Flask-based web application. This section also includes user authentication, account management, and how software sends alerts (using Fuzzy Logic and Threshold Evaluation). *Section 2.3 Conclusions:* This final section summarizes all the components introduced in *Chapter 2* and prepares for the system implementation and evaluation that will follow in later chapters, *Chapter 3: Results and Evaluation*.

2.1. Hardware System Architecture

This section introduces the hardware components used at both the *Sensor Node* and the *Central Processing Unit*.

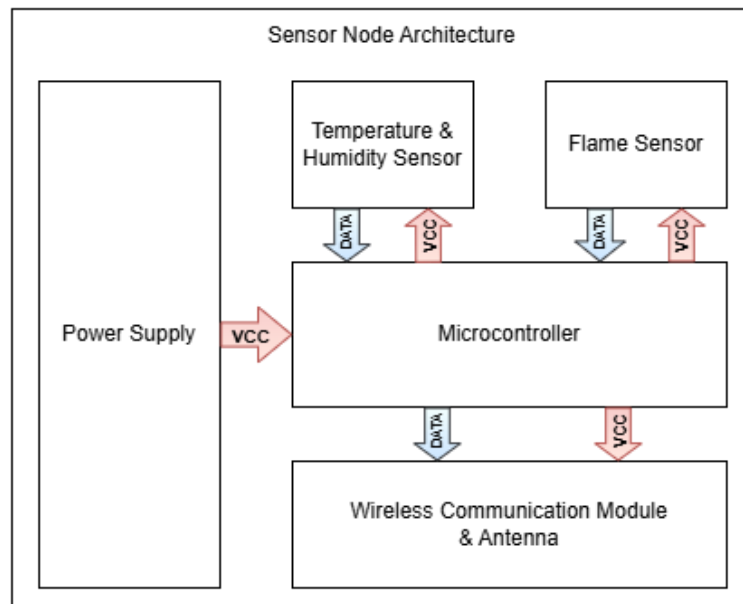


Figure 3: Sensor Node Architecture.

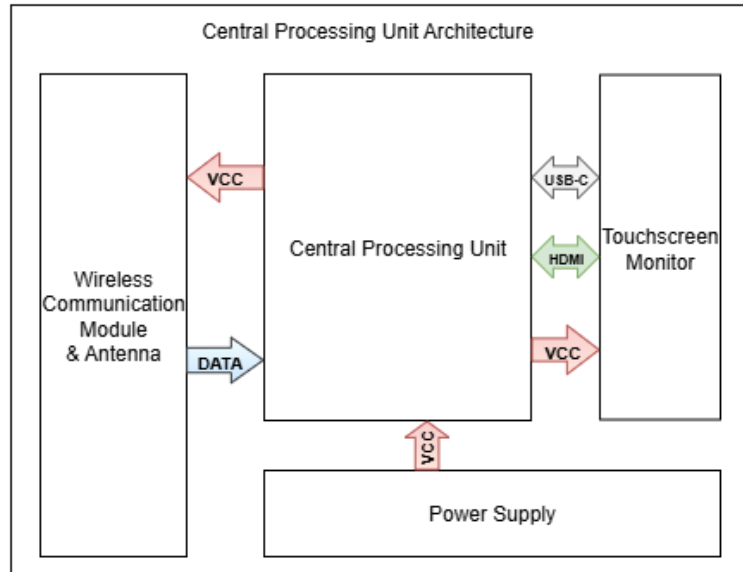


Figure 4: Central Processing Unit Architecture.

It explains what each component is, its role in the system, and the reasons behind choosing it for the demo. Subsections cover specific components, including fire sensors, temperature and humidity modules, STM32 for edge processing, and the Raspberry Pi for central control and web server hosting. The following *section 2.2* will discuss the data flow in depth, and how each part works together.

2.1.1. Sensor Node Architecture

(1) Flame Sensor using KY-026

Flame sensors are devices used to detect the presence of fire by sensing specific types of radiation emitted by flames, especially infrared light in the 760 – 1100 nm range. In this thesis, the KY-026 flame sensor is selected due to its low cost, small size, and ease of use. Despite being a basic sensor, it is well-suited for demo and educational purposes, making it a practical choice for the project's goals and budget.



Figure 5: KY-026 – Flame sensor [11].

(2) Temperature & Humidity Sensor using DHT11

A temperature and humidity sensor is used to collect environmental data, which is important in fire detection since fires typically cause a quick increase in temperature and a quick decrease in humidity. Depending only on a flame sensor may cause inaccurate results, so combining it with temperature and humidity will improve detection reliability and reduce false alarms. Although there are many types of environmental sensors, this thesis uses the DHT11 due to its low cost, low power, and availability, making it ideal for the demo goals of the project and suitable for academic applications.

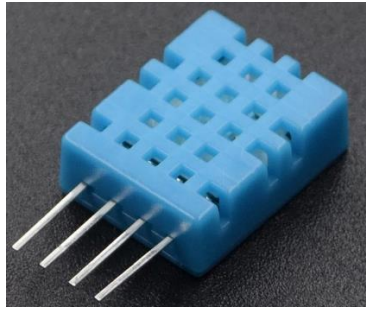


Figure 6: DHT11 - Temperature & Humidity Sensor [12].

(3) Microprocessor using STM32F103C8T6

In an IoT system, the edge processor is a lightweight microcontroller responsible for collecting data from sensors such as flame, temperature, and humidity sensors. This thesis uses the STM32F103C8T6 as the edge processor due to its suitability for the project's research and application goals. Its UART communication interface enables efficient serial data handling and transmission, making it a reliable choice for processing and transmitting sensor data in the proposed system.

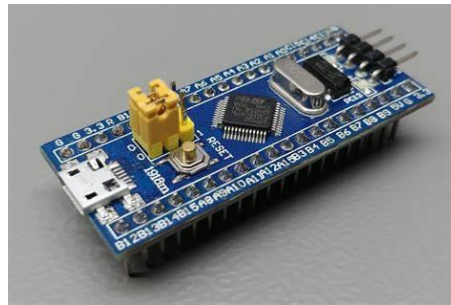


Figure 7: STM32F103C8T6 Microcontroller [13].

(4) Wireless Communication Module using HC-12

The wireless communication module is crucial in the proposed IoT system, enabling data transmission between sensor nodes and the central processing unit. While technologies like Wi-Fi, Zigbee, and Cellular require existing infrastructure, LoRa offers long-range communication with higher costs and complexity. In contrast, the HC-12 module provides a low-cost, easily configurable alternative well-suited to the project's goals. As described in Section 2.1.1, the HC-12 is integrated into the transmission block, where it wirelessly sends sensor data, processed by the STM32F103C8T6 microcontroller, through RF communication.

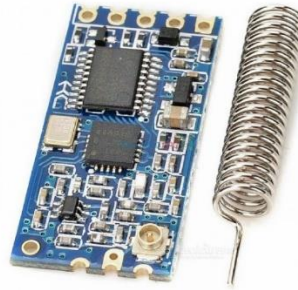


Figure 8: HC-12 – Wireless Communication Module [14].

(5) Antenna

To improve wireless performance, a 433 MHz external antenna is connected to the HC-12 module via an SMA male connector. With a 30 dBi gain, 50 Ω impedance, $SWR \leq 1.5$, and a height of 205 mm, the antenna enhances RF range and stability. It supports up to 100 W of input power and enables reliable, long-range communication in both indoor and outdoor environments, without relying on existing infrastructure.



Figure 9: 433 MHz RF Antenna [15].

(6) Sensor Node Schematic

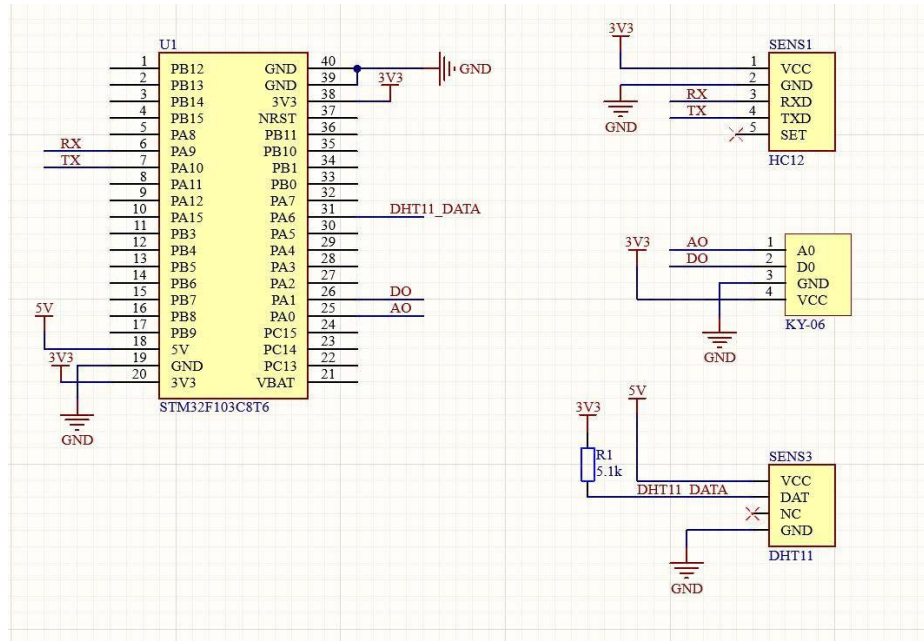


Figure 10: Sensor Node Circuit Diagram.

Using sensors with an edge processor on a test board is unsuitable because it does not provide a continuous and dependable supply of power to the sensors. To solve this issue, the thesis focuses on creating a specialized Printed Circuit Board (PCB) for the Sensor Node that improves electrical stability and performance (Figure 10).

2.1.2. Central Processing Unit Architecture

(1) Wireless Communication Module using HC-12

In the 2.1.2 Central Processing Unit Architecture, the HC-12 module is placed in the receiver (Rx) block. After the data is transmitted from the node to the central processing unit via radio communication, the HC-12 module at the receiver receives the incoming signals and transmits them to the central processor for processing.

(2) Central Processing Unit using Raspberry Pi 5

In this system, the central processing unit handles incoming data received wirelessly via the HC-12 module. To meet the project's goals, a Raspberry Pi 5 is used for its practical application in real-world IoT scenarios. As a single-board computer (SBC), the Pi 5 runs full operating systems like Raspberry Pi OS or Ubuntu and supports microSD storage up to 1 TB. Powered by a quad-core ARM Cortex-A76

processor and equipped with 8 GB of RAM, it delivers strong performance comparable to standard PCs. It also offers many connectivity options, including USB, Ethernet, Wi-Fi, Bluetooth, micro HDMI, and a 40-pin GPIO header for interfacing with sensors and modules like HC-12. With its power, flexibility, and ease of integration, the Raspberry Pi 5 is well-suited for modern IoT applications.



Figure 11: Raspberry Pi 5 – Single board Computer[16].

(3) Touchscreen Monitor using ASUS ZenScreen

Although this project does not focus on physical user interfaces, adding a touchscreen in the demo setup improves system visualization and user interaction. Therefore, an ASUS [17] ZenScreen touchscreen is used. It is fully compatible with the custom monitoring software developed in this project. This addition enhances user understanding, simplifies testing, and provides a basis for future improvements.



Figure 12: ASUS ZenScreen Touchscreen [18].

2.2. Software & Firmware System Architecture

Section 2.1 focuses on introducing the hardware components used in this thesis, along with the reasons for their selection and their unique benefits. *Section 2.2. Software & Firmware System Architecture* discusses the working flow of the full system implementation. *Section 2.2* is divided into four subsections: *Section 2.2.1.*

Data Processing at the Sensor Node (The Transmitter) explains how data is handled at the sensor node after collecting data from flame, temperature, and humidity sensors. It also provides the standard requirements for protecting the privacy and security of data during wireless transmission. *Section 2.2.2. Data Processing at Central Processing Unit (The Receiver)* uses software to receive, decode, and process incoming data packets. The decrypted data will be uploaded in real-time onto a web-based platform. *Section 2.2.3. Software Development at Central Processing Unit* covers software interface, such as user administration, data logging, notification handling, and other system operations. *Section 2.2.4 System Alert* discusses system alerts, warnings triggered by thresholds, and the use of Fuzzy Logic. All content of the section will be described in further depth.

2.2.1. Data Processing at the Sensor Node (The Transmitter)

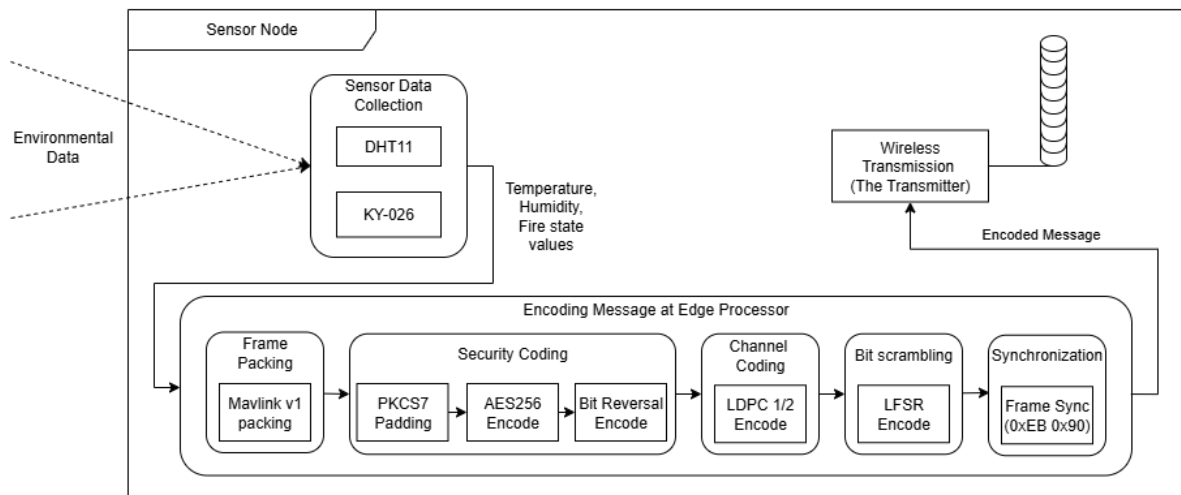


Figure 13: Data Processing at the Sensor Node.

(1) Sensor Data Collection

Input: Value from the environment.

Output: Temperature, Humidity, Fire state data.

During the Sensor Data Collection stage, environmental data is collected using two sensors: the DHT11 for temperature and humidity, and the KY-026 for fire detection.

Algorithm 1: Read DHT11 Data and Fire Sensor State.

```
Loop forever  
    start DHT11, confirm Presence (check DHT11 response)  
    Rh_byte1, Rh_byte2, Temp_byte1, Temp_byte2, crc  $\leftarrow$  read  
    five bytes  
    valid  $\leftarrow ((\text{Rh\_byte1} + \text{Rh\_byte2} + \text{Temp\_byte1} + \text{Temp\_byte2})$   
     $\& 0xFF) = \text{crc}$   
    Temperature  $\leftarrow \text{float}(\text{Temp\_byte1})$   
    Humidity  $\leftarrow \text{float}(\text{Rh\_byte1})$   
    Fire  $\leftarrow \text{readGPIO}(\text{GPIOA}, \text{PIN1})$   
end loop
```

The DHT11 employs a 1-Wire interface, with the STM32 initiating communication through GPIOA Pin 6 set as output with accurate time delays. After setting up, it returns 5 bytes (humidity, temperature, and checksum), and data validity is validated by checking the checksum. The KY-026 flame sensor, which is attached to GPIOA Pin 1 (digital) and, alternatively, GPIOA Pin 0 (analog via ADC), produces HIGH (1) when no flame is detected and LOW (0) when there is a fire.

(2) Encoding Message

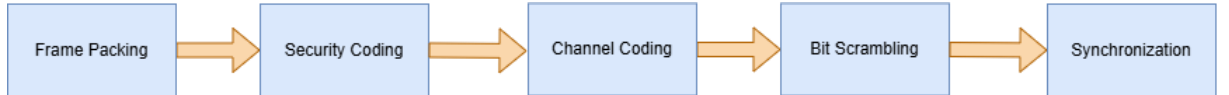


Figure 14: Data protection sequence in wireless transmission.

This is a highly important and interesting part when transmitting messages over wireless communication. It ensures compliance with standard requirements for protecting privacy and securing data in wireless transmission. In wireless data security, the main components typically include (Figure 14): Frame Packing, Security Coding, Channel Coding, Bit Scrambling, and Synchronization.

a) Frame Packing using MAVLink v1

Input: Temperature, Humidity, Fire state data.

Output: MAVLink frame.

In the Frame Packing step of the thesis, sensor data – including temperature, humidity, and fire state – is packed into a standardized communication format using the Micro Air Vehicle Link (MAVLink [19]) protocol version 1. This protocol, widely used in embedded and wireless systems, ensures reliable and efficient data exchange. Each MAVLink frame comprises three sections: a 6-byte Header, a Payload of up to 255 bytes, and a 2-byte CRC.



Figure 15: MAVlink v1 Frame [20].

As in Figure 13, the Header includes the start byte (0xFE), payload length, a message sequence number, system ID, component ID, and message ID. The Payload carries the original sensor values, while the CRC is used to verify data integrity and detect transmission errors. This structure enables robust, consistent communication between the nodes and the CPU in the fire monitoring system.

b) Security Coding using AES-256 in mode ECB

Input: MAVLink frame.

Output: Ciphertext.

In the Security Coding step, to enhance the protection of MAVLink. This thesis applies the AES-256 [21] - Block encryption algorithm, one of the most powerful and widely adopted encryption standards today. With a 256-bit key length and 14 rounds of transformation, AES-256 provides strong resistance against modern cryptographic attacks. Before using AES-256, PKCS7 [22] - Padding is used to make a multiple of 16. After padding, the message is divided into fixed-length blocks and encrypted sequentially with AES-256. Additionally, to further confuse the data, this thesis uses the bit reversal, which reverses all the bits in each block.

c) Channel Coding using LDPC 1/2

Input: Ciphertext.

Output: Channel-encoded message.

At this stage, Low-Density Parity-Check Code 1/2 (LDPC 1/2) is used with a shared parity check matrix H to correct error bits during transmission over a radio frequency channel. This step improves transmission reliability and integrity when the transmitter operates in harsh environments.

d) Bit scrambling using LFSR Fibonacci

Input: Channel-encoded message.

Output: Scrambled message.

In the Bit Scrambling phase, this thesis uses a Fibonacci-type Linear Feedback Shift Register [23] (LFSR) to eliminate long sequences of consecutive 0s or 1s in the bitstream. This helps prevent data loss and improves overall communication reliability and security. By generating pseudo-random sequences, the LFSR scrambles the signal, making it more resistant to detection, tracking, and potential attacks during wireless transmission. At this stage, the input is an LDPC-encoded message whose length has been doubled through channel coding. Each data block is XORed with the LFSR output, using a shared register at both the transmitter and receiver to enable proper decoding.

e) Synchronization

Input: Scrambled message.

Output: Framed message.

After undergoing the processes of frame packing, encryption, channel coding, and bit scrambling, the message becomes significantly confused, making it difficult for the receiver to identify the beginning or structure of the transmission. Therefore, as the final step, two header bytes (0xEB and 0x90) are added after LFSR encoding.

These specific bytes were chosen because 0xEB and 0x90 are commonly detectable in wireless environments. They frequently appear in the radio spectrum, making them reliable markers. By adding these two bytes at the beginning of the final message, the receiver can more easily synchronize and detect the start of a valid

transmission. In the Encoding Message section, each component will be described in detail through corresponding steps, providing a clear understanding of how the message is processed and prepared for transmission.

(3) Wireless Transmission (through UART – The Transmitter)

Input: Framed message.

Output: Message transmitted.

In the Wireless Transmission stage, the framed message is sent from the STM32 to the HC-12 module using UART communication. The STM32 delivers the data over its UART interface, while the HC-12 module handles modulation and RF transmission. To ensure communication, both the STM32 and HC-12 must be configured with the same baud rate of 115200 and operate on the same channel. These parameters are set via AT commands during the initialization process. Once configured, the HC-12 broadcasts the message wirelessly, completing the transmission process.

2.2.2. Data Processing at Central Processing Unit (The Receiver)

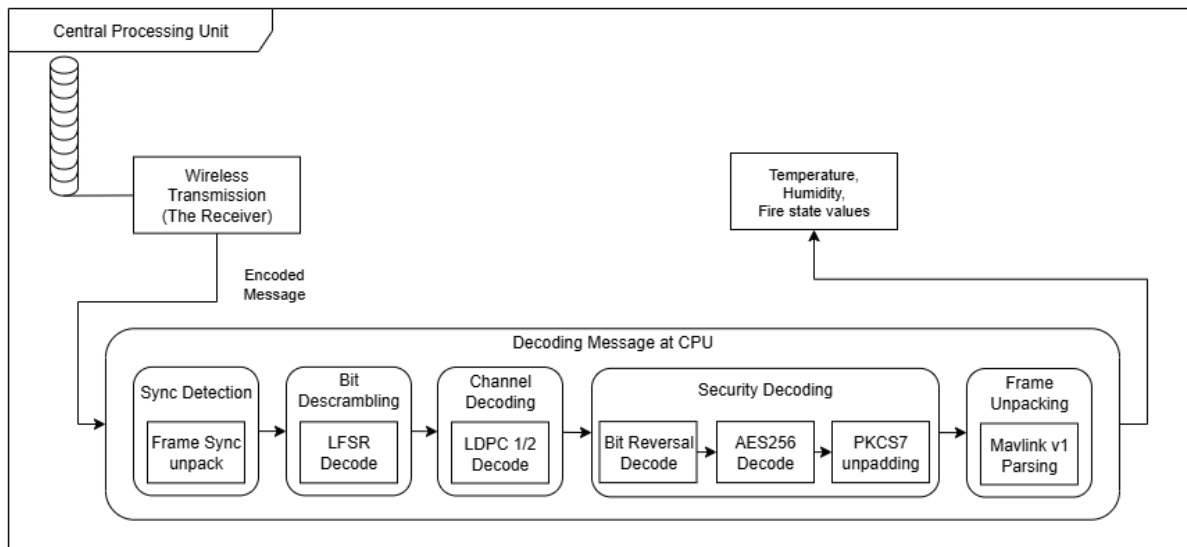


Figure 16: Data Processing at the Central Processing Unit.

(1) Wireless Receiver

Input: Message transmitted.

Output: Message received.

On the receiver side, the HC-12 wireless module is pre-configured with the same AT command parameters, such as baud rate and channel, as described in the **Wireless Transmission** section. After the message is successfully transmitted, the CPU listens for incoming data through the USB UART.

(2) Decoding Message



Figure 17: Flow Decoding message at the CPU.

This is the process of decoding a message, which is the opposite of encoding.

a) Sync Detection

Input: Message received.

Output: Frame message unpack.

When the message is received through USB UART at the Central Processing Unit, it checks the validity of the message by inspecting the first two bytes. If the first two bytes are 0xEB and 0x90, the message is valid. The synchronization decode step then unpacks the frame by removing these two header bytes, allowing the system to proceed with further processing.

b) Bit Descrambling

Input: Frame message unpack.

Output: Message descrambling.

Because both the transmitter and the receiver use the same register after applying the LFSR [23] Fibonacci, to decode the bit scrambling, XOR each block again with the LFSR Fibonacci output, just as in the LFSR encoding step.

c) Channel Decoding

Input: Message descrambling.

Output: Channel decode message.

At this stage, LDPC [24] 1/2 decoding is employed to detect and correct errors caused by wireless transmission. As said in Channel Coding, a shared parity-check matrix H is applied to compute the syndrome. If the syndrome vector contains non-zero values, it indicates the presence of errors. The decoder then tries to correct the bit error by finding and flipping the bit related to the most incorrect parity. The syndrome is recalculated using the calculate syndrome function after each bit flip, and this process continues until the syndrome is zero or a maximum of ten iterations (the number of iterations is user-defined) is reached. If problems continue after the allowed rounds, the message becomes uncorrectable and is destroyed.

d) Security Decoding

Input: Channel decode message.

Output: Plaintext.

After LDPC 1/2 decoding, if the syndrome vector contains only zeros, the message is considered error-free and proceeds to the Security Coding Decode stage. As mentioned earlier, bit masking in this system involves a simple bit-reversal operation during encoding, so decoding applies bit reversal again to restore the original bit order, preparing the message for AES-256 [21] decryption and PKCS7 [22] unpadding.

In the AES-256 decryption phase, the same key used during encryption is reused, but the round keys are applied in reverse order, from the last to the first, across 14 decryption rounds. Once decryption is complete, the message enters the PKCS7 unpadding stage, where the decoder checks the last byte to determine the padding length and removes the corresponding number of padding bytes. If the padding is invalid, the message is discarded. After unpadding successfully, the original MAVLink frame is fully recovered.

e) Frame Unpacking

Input: Plaintext.

Output: Temperature, Humidity, Fire state data.

In this final stage, the system verifies whether the plaintext corresponds to a valid MAVLink frame [19] . If the first byte is 0xFE, it indicates the start of a MAVLink packet. The decoder compared the new CRC, which is recalculated from byte 2 onward (excluding the start byte, 0xFE), with the received CRC. If two CRCs match, the frame is valid; otherwise, it is discarded. Once validated, the payload is extracted to obtain the final output, which includes Temperature, Humidity, and Fire status data.

2.2.3. *Software Development at Central Processing Unit*

This section presents the implementation of the software system on a web platform, aiming to improve user interaction, usability, real-time monitoring, and overall system management.

a) Software Architecture

Software Design

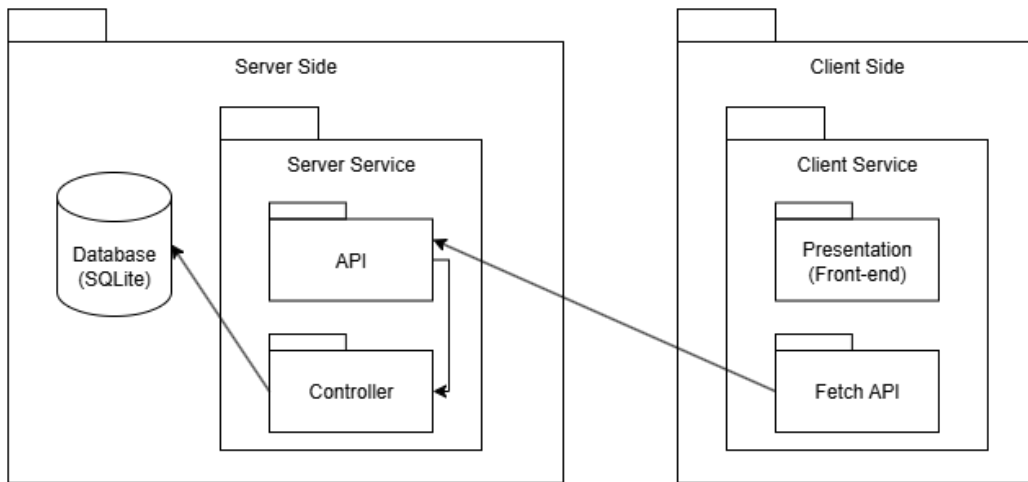


Figure 18: Software Architecture diagram.

The system architecture comprises several key components that work smoothly between the server and client sides. On the server side, a **database** serves as the central repository for user data, activity logs, fire alerts, and settings. A **server service** manages client requests and data access, while **APIs** act as communication bridges, handling requests, executing logic, and delivering functionalities. **Controllers** implement business logic, coordinating data flow between APIs and the database. On the client side, the **user interface**, built with HTML, CSS, and JavaScript, enables dynamic interaction by calling server APIs. Using the **Fetch API**, the client can send

requests, allowing smooth, real-time updates without reloading the entire page, thereby enhancing performance and user experience.

Tool & Technicals

The software system applies the MVC (Model – View – Controller) architecture, including Controller, Service, Repository, and Model layers for a clean and maintainable structure. The backend is developed using Flask, a lightweight Python framework that is ideal for interacting with GPIO and communicating through UART with the Edge Processor. SQLite is chosen as the database for its simplicity and seamless integration with Flask, making it suitable for embedded applications.

The frontend is built with HTML, CSS, and JavaScript, providing a responsive and interactive user interface. APIs bridge the frontend with the backend, enabling real-time data fetching and updates. Because the system lacks a continuously online server, a VPN solution (TailScale) is employed to allow remote access. TailScale offers secure peer-to-peer connectivity, making it a suitable choice for IoT systems that require remote monitoring and control without dedicated server infrastructure.

b) Requirement analysis

Functional requirement

The following table outlines the functional requirements of the system, based on the roles of the two main actors: Admin and User.

ID	Functionality		Description	Roles
1	Maintain User Information	Sign Up	Register a new account	Admin, User
		Password Recovery	Recover forgotten password	
		Login	Log in to the system	
		Edit Personal Information	Edit username, password, email, and phone number	
2	Maintain Dashboard	View Log & Fire Events	View daily logs and fire event history	
		Real-time Data Monitoring	View real-time sensor data on the dashboard	

ID	Functionality		Description	Roles
3	Export logs		Export the data for the days	
			Export fire events	
4	User Management	Create a new user	Add user accounts	Admin
		Delete exist user	Remove user accounts	
		Approval Sign Up queue	Approval Sign Up queue	
5	Maintain Alert & Threshold	Alert Deactivation & Notification	Send an alert if a fire is detected or the alert is deactivated	
		Threshold Settings	Modify temperature and humidity thresholds and notify all users	

Table 1: Functional requirement.

Non-functional requirements

- **Performance:** The system must show sensor data (temperature and humidity) on the dashboard within 2 seconds after receiving it from the STM32 node.
- **Wireless Transmission Reliability:** Since the system uses HC-12 wireless modules, it must detect and handle possible transmission errors to keep data accurate and complete.
- **Data Processing Efficiency:** The process of data from node to central processing unit - encode and send sensor data, then decode each message in under 1 second.
- **Data Security:** All data sent wirelessly must be encrypted to prevent tampering. User passwords must be securely hashed using reliable algorithms.
- **Email Alert Delivery:** The system must send email alerts reliably, with at least a 98% success rate when a fire is detected or thresholds are exceeded.
- **Scalability:** The system should handle multiple sensor nodes at the same time without slowing down or becoming unresponsive.
- **Cross-Platform Compatibility:** The web application must work well on major browsers like Chrome, Firefox, Microsoft Edge, and others.

- **Reliability:** Even though wireless transmission may sometimes fail, the system should remain reliable by using error checking or upgrading the wireless module if needed.

- **Usability:** The system should be easy to use. Users should be able to view sensor data, receive alerts, and export logs without needing technical knowledge.

c) System analysis and design

Use case diagram

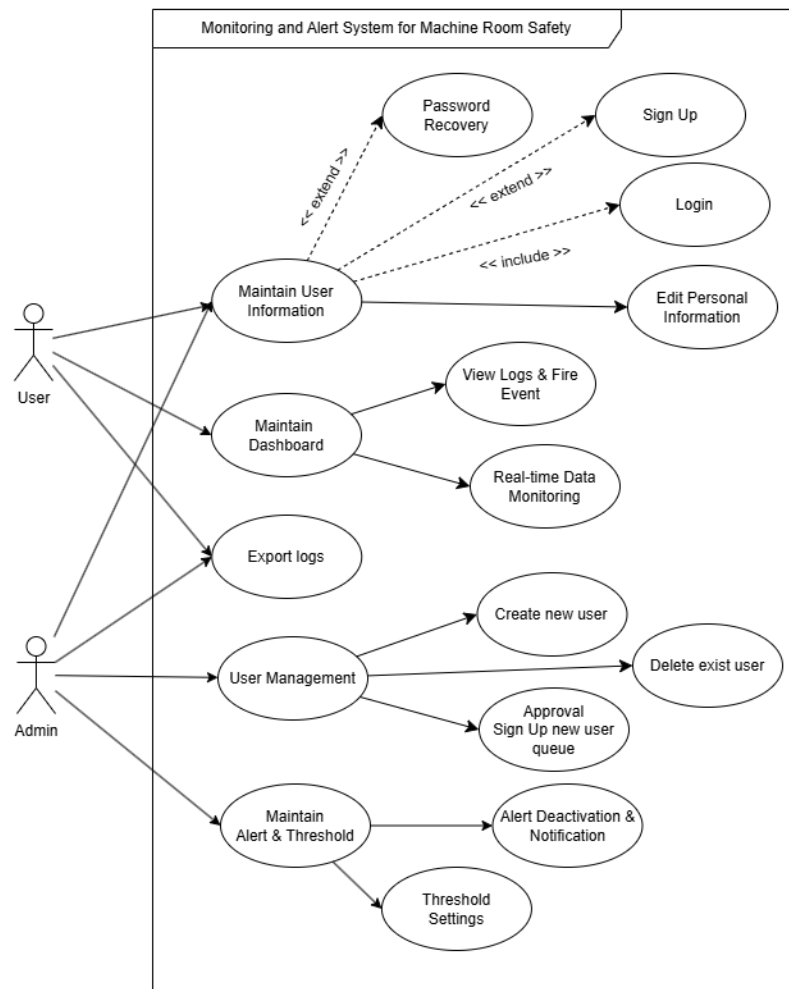


Figure 19: Use case diagram.

The use case diagram shows two main actors of the system:

- **User:** A regular user who can sign up, log in, update personal information, monitor real-time data, receive alerts, and export logs.
- **Admin:** A superior user who manages accounts, approves sign-up user requests, sets alert thresholds, and sends system notifications.

The system’s core functionalities are grouped into Maintain User Information, Maintain Dashboard, Export logs, User management, and Maintain Alert & Threshold.

Sequence Diagrams

Export Logs

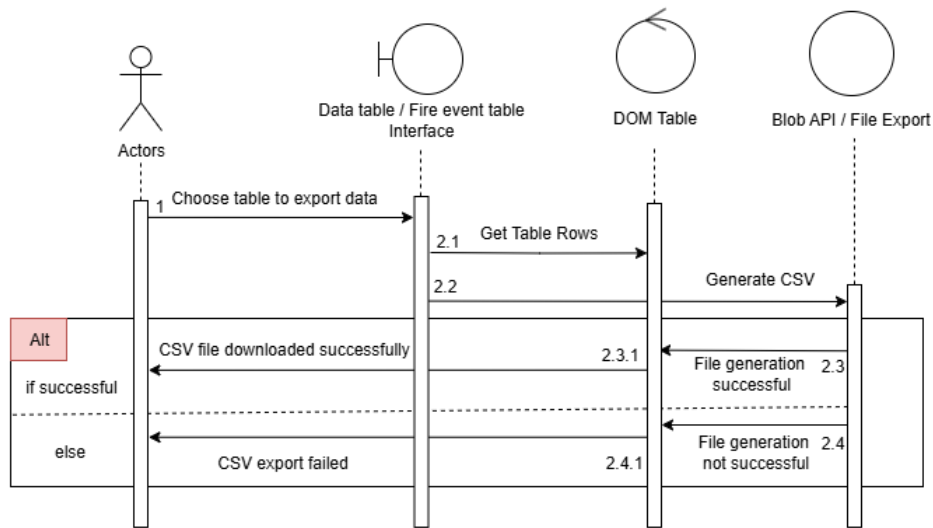


Figure 20: Export Logs Sequence Diagram.

This sequence diagram illustrates the process of exporting tabular data upon user request. The user can choose to export either daily monitoring data or fire event logs. The request is initiated via the client interface, processed through a controller, and handled by the service layer. Depending on the type selected, the system queries the corresponding data set from the SQLite database, processes it into a downloadable format (e.g., CSV), and returns the result to the client. This functionality provides flexible data access and supports user needs for reporting, analysis, or archival.

Threshold Settings

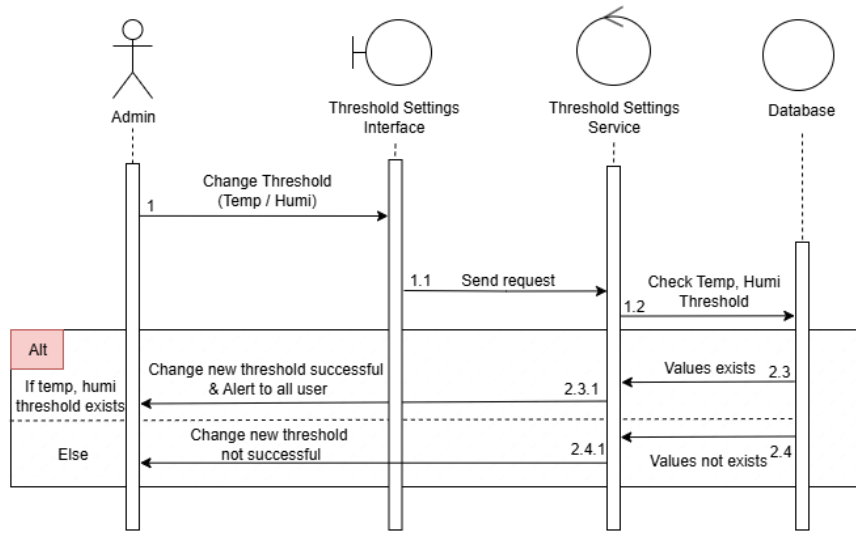


Figure 21: Threshold Setting Sequence Diagram.

This function enables the Admin to adjust the threshold values used to trigger alerts to users. The threshold includes two parameters: temperature and humidity.

Although the DHT11 sensor can theoretically return values up to 50°C and 20% humidity, using values too close to these limits may reduce the sensor's reliability and negatively impact the alert system. Therefore, in this thesis, the thresholds are intentionally set at 40°C to 45°C for temperature and 30% to 35% for humidity to ensure both accurate alerts and the long-term safety of the sensor.

Deactivate Alert

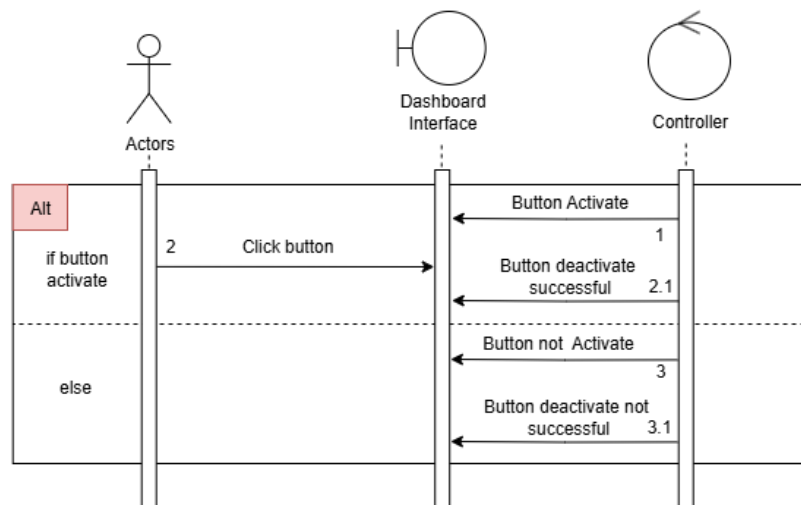


Figure 22: Deactivate Alert Sequence Diagram.

When an alert is triggered in the System Alert section (detailed in the section below), a button to dismiss the alert will appear on the Admin's dashboard. The Admin is authorized to deactivate this button, which records the end time of the fire or alert event at the time. Once the button is deactivated, an email notification will be sent to inform users that the event has ended.

2.2.4. System Alert

This is an essential part of the system that handles user notifications. This section contains two parts: the Threshold-based Alert Mechanism using Temperature Threshold, Humidity Threshold, and the second part, the Fuzzy Logic-based Alert Mechanism using Fuzzy Logic.

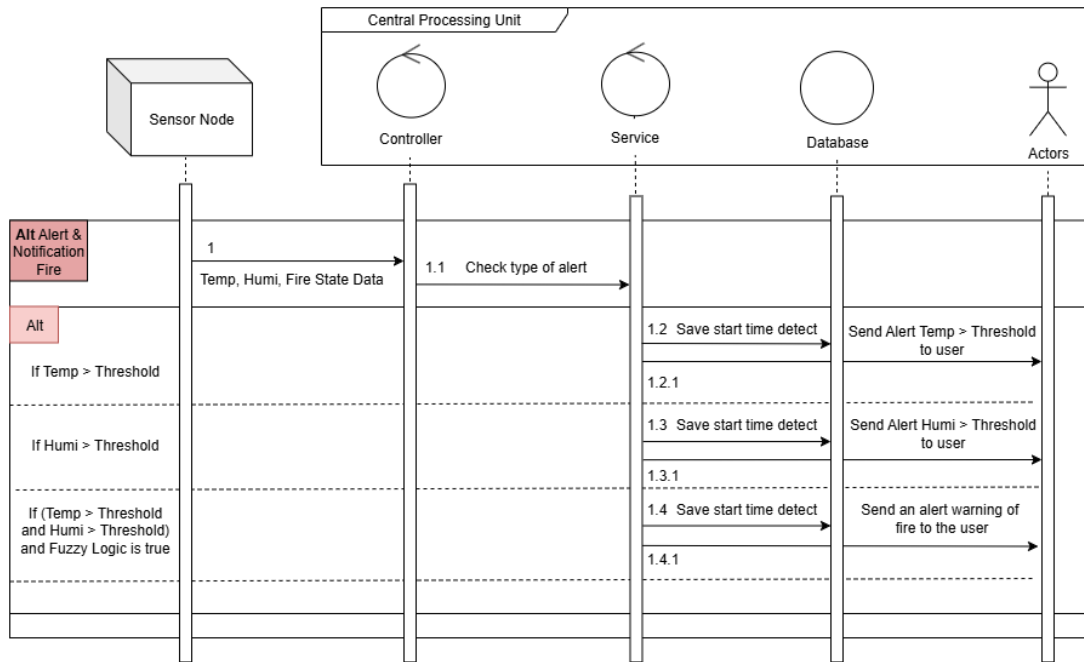


Figure 23: Alert Sequence Diagram.

a) Threshold-based Alert Mechanism

Temperature & Humidity Threshold Alerts

In this section, the system determines fire alarms based on the sensor's capabilities. The DHT11 sensor operates within a temperature range of 0-50°C and a humidity range of 20-90%. However, to prevent hardware damage and enable early fire detection, this thesis does not wait for the maximum temperature to activate

alarms. Instead, an alert occurs when the temperature reaches 40°C and the humidity drops below 30%. In both cases, an email notification is sent to all users.

b) Fuzzy Logic-based Alert Mechanism

Fuzzy Logic [25] is a data processing approach based on approximate reasoning, allowing values to range between 0 and 1, rather than being completely binary (true = 1 or false = 0) as in traditional logic systems. Instead of assigning a crisp value like “Cold” (0) or “Hot” (1), Fuzzy Logic introduces transitional states—for example, a value in between, such as “Warm,” is represented by a degree of membership between 0 and 1.

As illustrated in the figure below, this flexible representation enables the system to evaluate the changes and generate risk levels ranging from low to high. Consequently, it allows for more nuanced alerts and earlier warnings, rather than waiting until a hard threshold is reached.

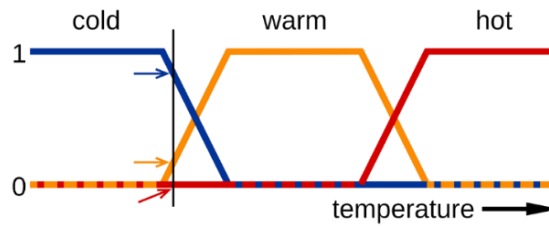


Figure 24: Fuzzy Membership Functions for Water State (Cold, Warm, Hot).

There are various types of Fuzzy Logic systems, including Rule-Based, as well as more advanced variants such as Learning-Based, Adaptive, and other forms like Fuzzy Weighted Sum.

However, this thesis adopts the Rule-Based Fuzzy Logic approach, which is also the most fundamental and representative form of fuzzy reasoning. The rule-based model consists of three main steps: **Fuzzification** transforms all input values into fuzzy membership functions. **Rule Evaluation** – apply all applicable rules from the rule base to compute the fuzzy output sets. **Defuzzification** – convert the fuzzy output sets into crisp output values. This approach is chosen for its simplicity, interpretability, and effectiveness in modeling human-like reasoning in fire detection scenarios.

Fuzzification:

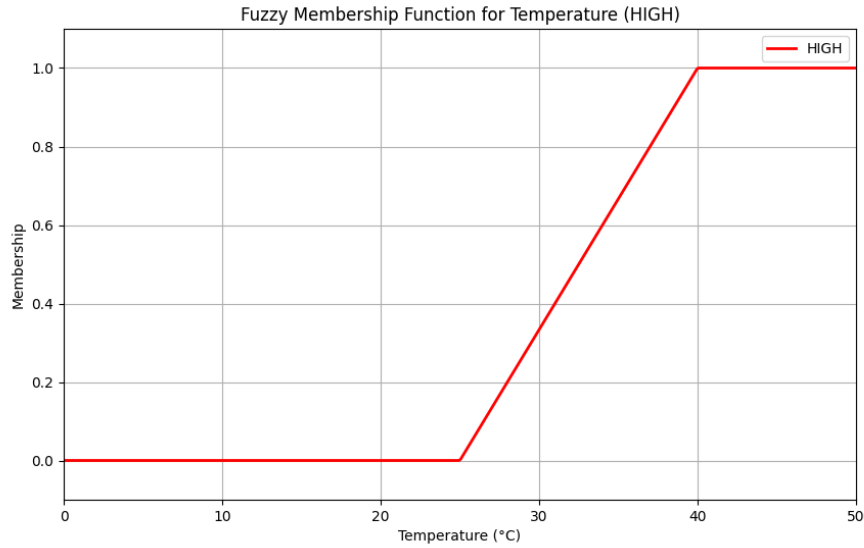


Figure 25: Membership of Temperature.

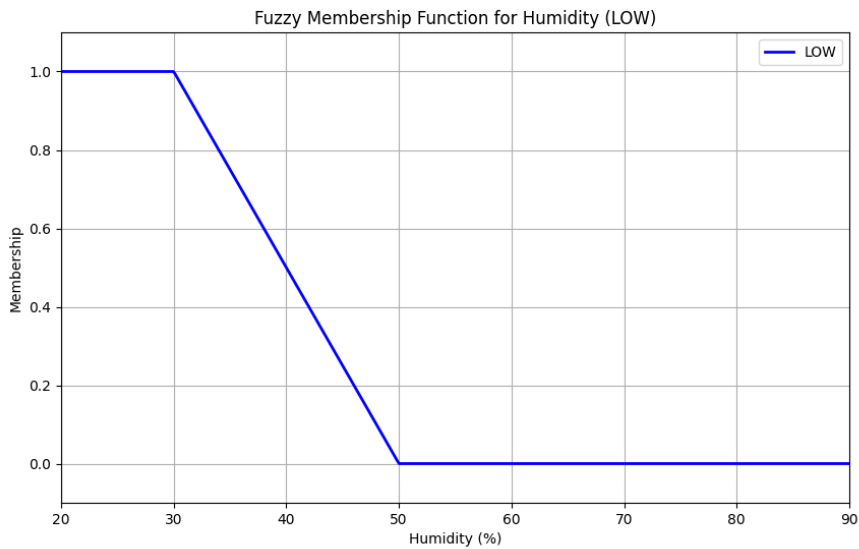


Figure 26: Membership of Humidity

To simplify the initial development process, this thesis only uses two major membership functions: Temperature High and Humidity Low. Limiting the model to these two levels, rather than adding levels such as Low, Medium, or High for each variable, helps to reduce system complexity and prevent rule explosion. Using a larger number of fuzzy rules adds complexity, potentially leading to inconsistencies and decreased accuracy. The system's primary purpose is to detect fires quickly and

reliably. As a result, the model focuses on the most important conditions commonly connected with the fire:

- High temperature: Fire causes an increase in ambient temperature. In this model, temperature is considered to gradually become “High” starting at 25°C, reaching “Fire” (1) at 40°C and above.
- Low humidity: A dry environment provides ideal conditions for fire to spread. So, humidity below 30% is considered an important risk, while levels above 50% are as safe.

This simplification enables a more efficient fuzzy rule base design without significantly compromising performance. Although the model does not account for intermediate states like Medium, it brings a high effective value to concentrate on detecting clearly high-risk fire conditions.

Rule Evaluation:

Rule	Fire status	Temperature	Humidity	Output Level
R1	Yes (1)	High (> 0.5)	Low (> 0.5)	VERY HIGH
R2	Yes (1)	High (> 0.5)	Not Low (≤ 0.5)	HIGH
R3	Yes (1)	Not High (≤ 0.5)	Low (> 0.5)	MEDIUM
R4	No (0)	High (> 0.5)	Low (> 0.5)	HIGH
R5	No (0)	Not High (≤ 0.5)	Not Low (≤ 0.5)	LOW

Table 2: Rule of Temperature, Humidity, Fire status.

The fuzzy system uses five rules to evaluate fire risk based on temperature, humidity, and fire detection status. Each rule is triggered according to the degree of membership in “high temperature” and “low humidity”. Rules with fire detected (fire state = 1) prefer higher alert levels, especially when both temperatures are high and humidity is low. Rules without fire serve as early warnings based on bad environmental conditions. Each rule contributes a weighted score based on severity, which is averaged to produce the final fuzzy risk level. This approach ensures flexible and realistic decision-making beyond fixed thresholds.

Defuzzification:

After evaluating all activated fuzzy rules, the system applies a weighted average method to perform defuzzification. Each output label (LOW, MEDIUM, HIGH,

VERY HIGH) is assigned a numeric weight reflecting its severity level (3, 5, 7, 9, respectively). The final fuzzy score is calculated by averaging these weights based on rule strength.

This crisp score is then used to analyze the fire risk level:

- A score above or equal 7 is considered a high probability of fire, triggering an immediate fire alert.
- A score between 4 and 7 show a potential fire risk, prompting early warnings.
- A score below or equal to 4 suggests the environment is safe.

This approach ensures that the system reacts appropriately and equally to environmental conditions, enhancing its reliability in early fire detection scenarios.

2.3. Conclusions

In the *Hardware System Architecture* section, the thesis introduced each component in detail, starting from the sensor nodes to the CPU, clearly defining and highlighting them. Next, in the *Software & Firmware System Architecture* section, the focus is on how the sensor nodes collect environmental data, then encode and transmit it through wireless communication (HC-12) to the central unit. At the CPU, the encoded packets received from HC-12 are decoded and parsed to extract meaningful values such as temperature, humidity, and fire status. A web-based interface was developed using a Flask web server to enable real-time data monitoring. The user-friendly website allows users to monitor temperature, humidity data, and fire state in real-time. Moreover, the web system includes essential user functionalities, such as login, signup, and profile management, among others. Admin can edit threshold values for alert conditions. Regarding the alert mechanism, the thesis implements a Fuzzy Logic-based algorithm to determine fire risk levels. All alert-related information is automatically sent to users through email, ensuring timely notification. In summary, Chapter 2 provides a detailed and cohesive overview of the entire Monitoring and Alert Fire System, from sensor collection and transmission to real-time visualization on the web interface and intelligent alert generation, highlighting both the hardware architecture and software functionality.

Chapter 3. Results and Evaluations

If Chapter 2 presented a detailed design of the system — from both hardware and software aspects – covering data collection from sensors, data encoding, transmission, decoding, processing, and real-time visualization, then Chapter 3 *Results and Evaluations* focuses on analyzing experimental outcomes to verify the effectiveness and reliability of the IoT-based fire monitoring and alert system. This chapter is divided into four main sections: *Section 3.1 Implementation*: Demonstrates the system hardware to software, all features for interacting with the user, and the system alert warning user, both of which have been successfully implemented and are operating smoothly. *Section 3.2 Reliability and Error Rates*: Evaluates the system’s reliability through analysis of error rates, packet loss, and the accuracy of false fire detections or failure to detect in situations. This section also measures the end-to-end delay from sensors to the user interface and alert notifications, to assess the system’s responsiveness in real time. *Section 3.3 Evaluation and Challenges*: Evaluate the system with a similar system and show the difficulties when deploying the system, and the practical applicability in real-world deployment. The last section, *Section 3.4 Conclusion*: Summarizes the findings presented throughout Chapter 3.

3.1. Implementation

Sensor Node Side

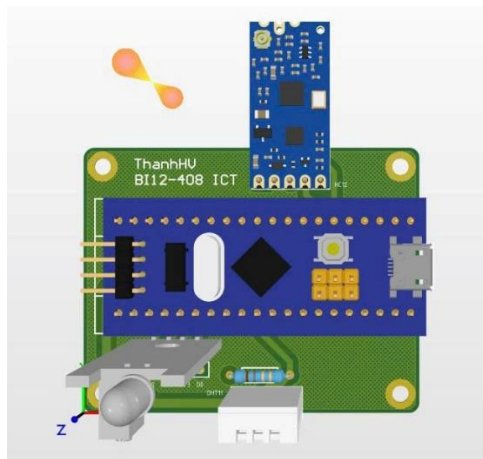


Figure 27: 3D Sensor Node Circuit Diagram.

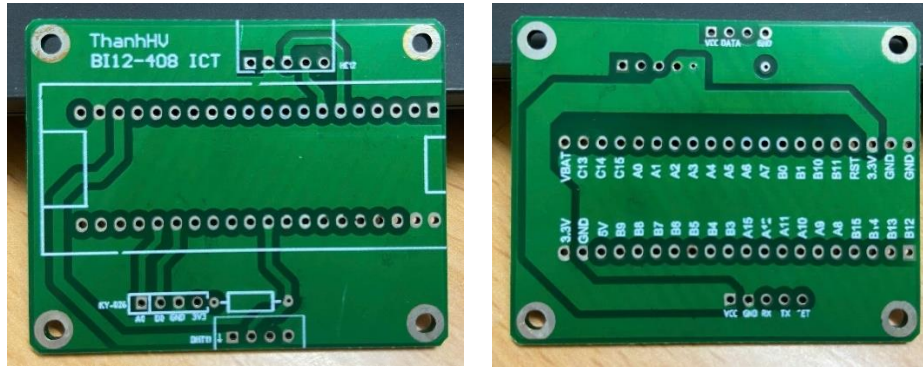


Figure 28: PCB for Sensor Node.



Figure 29: Edge Processor with various sensors and a wireless module.

On the sensor node side, as shown in the Sensor Node Schematic, a PCB was designed using Altium to implement the schematic-level design. The implemented PCB layout is presented in Figure 28, and the completed sensor node after building is shown in Figure 29. The STM32F103C8T6 microcontroller is directly soldered onto the PCB and connected to sensors, including the DHT11 temperature and humidity sensor, the KY-026 flame sensor, and the HC-12 wireless communication module. Additionally, an IPEX (u.FL) cable is used to link the wireless module to an external antenna for improved signal transmission.

Expression	Type	Value
DHT1	float	31
Humidity	float	73
rxBuf	float	Failed to evaluate expression
txBuf	float	Failed to evaluate expression
Fire	uint8_t	1 \\'001\''

Figure 30: Debug STM32 IDE Temp, Humi, Fire state value.

Next, for the firmware, the system utilizes STM32CubeIDE to implement code that collects environmental data. These sensors then render the output values for temperature, humidity, and fire state, as previously described. And, the fire state is by default defined as 1 when no flame is detected.



Figure 31: Sensor Node.

Finally, after the sensors collect the data, the next step is to encode and transmit it to HC-12. The HC-12 module is then connected to an antenna to enhance long-range signal transmission, enabling reliable communication with the CPU that is awaiting incoming data.

Central Processing Unit Side



Figure 32: Central Processing Unit.

At the Central Processing Side, after receiving data from the Sensor Node Side, it is decoded and parsed to extract the values for temperature, humidity, and fire state. The extracted data is then displayed in real-time on a user-friendly web-based platform, which also includes various functionalities as described in the *Software Development at Central Processing Unit* Section in Chapter 2 (see all figures below and in the Appendix).

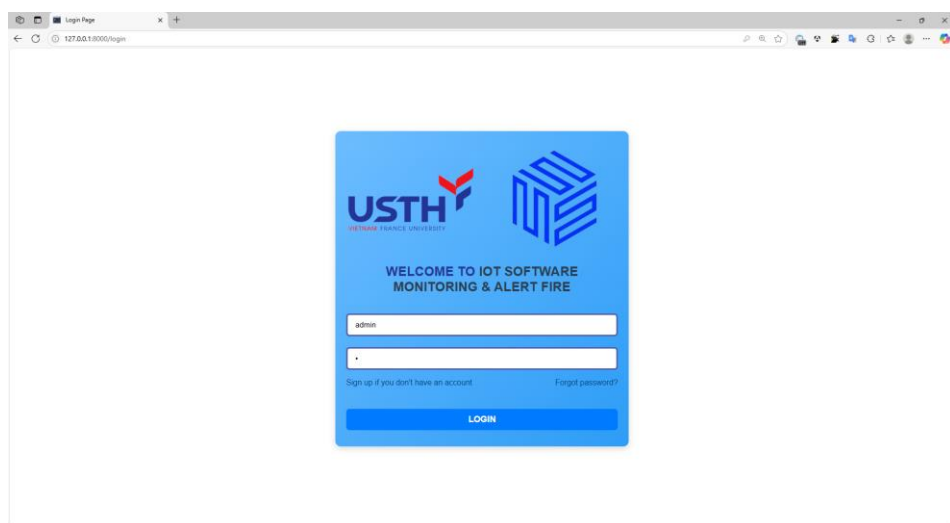


Figure 33: Software Interface – Login.



Figure 34: Software Interface.

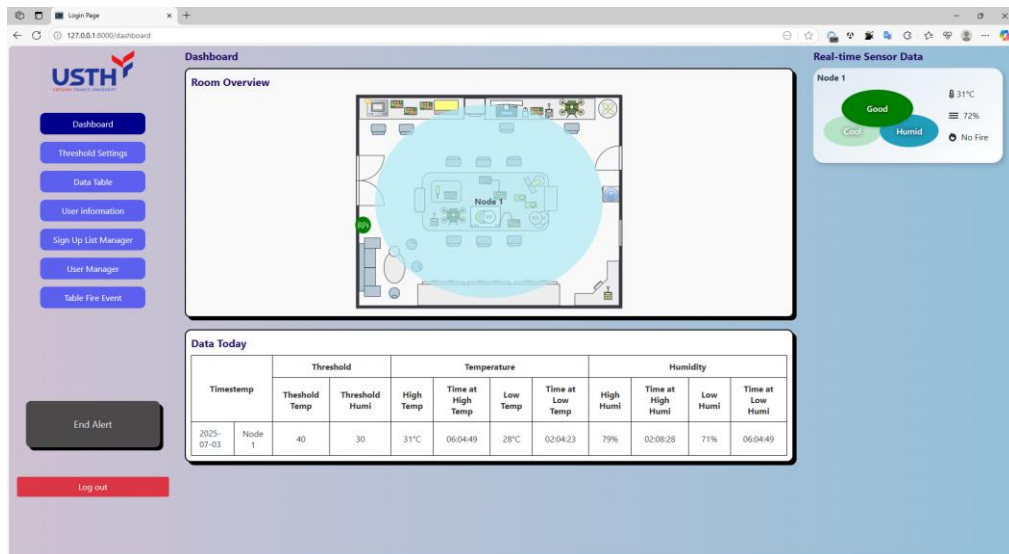


Figure 35: Software Interface – Dashboard.

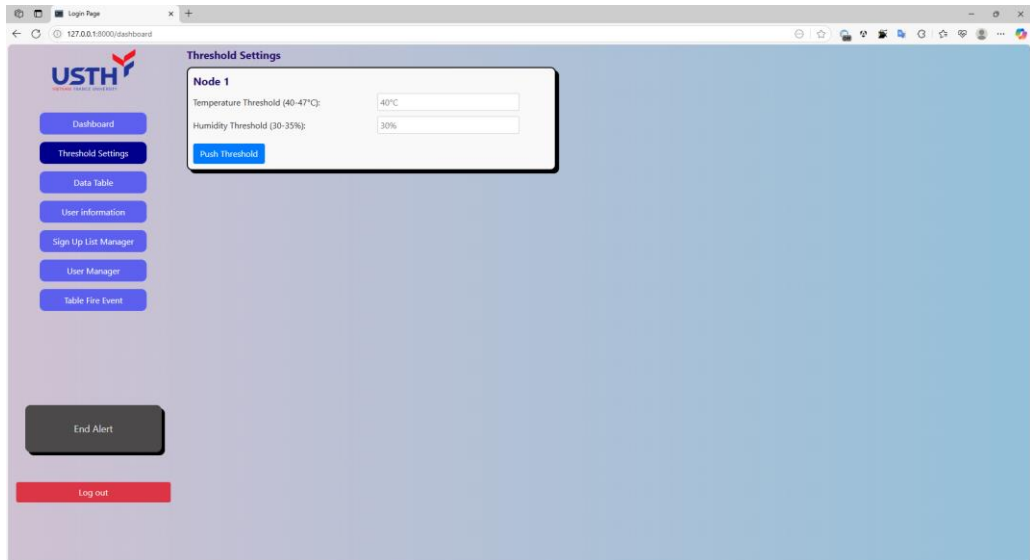


Figure 36: Software Interface – Threshold Settings.

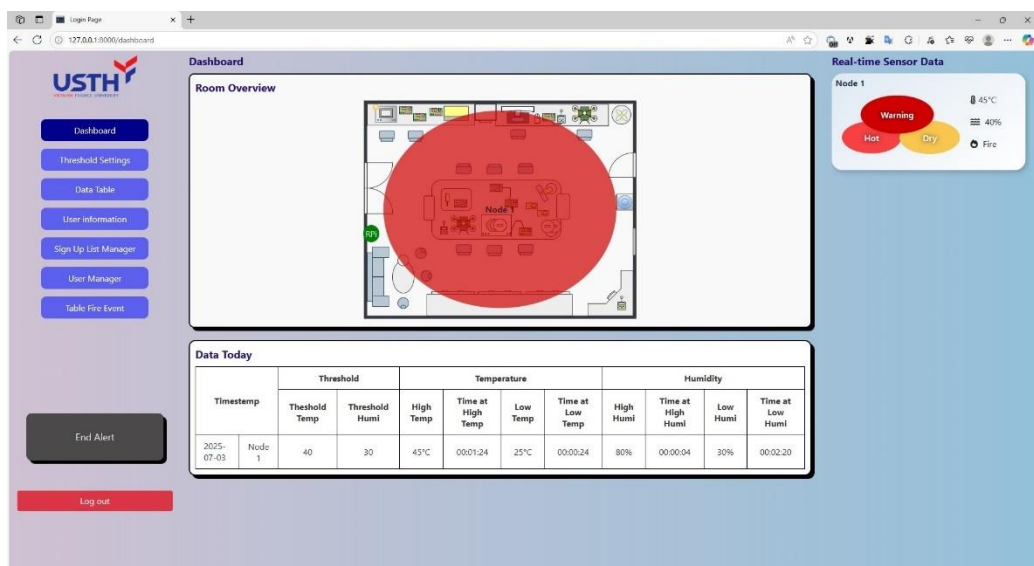


Figure 37: Software Interface – Dashboard (Fire).

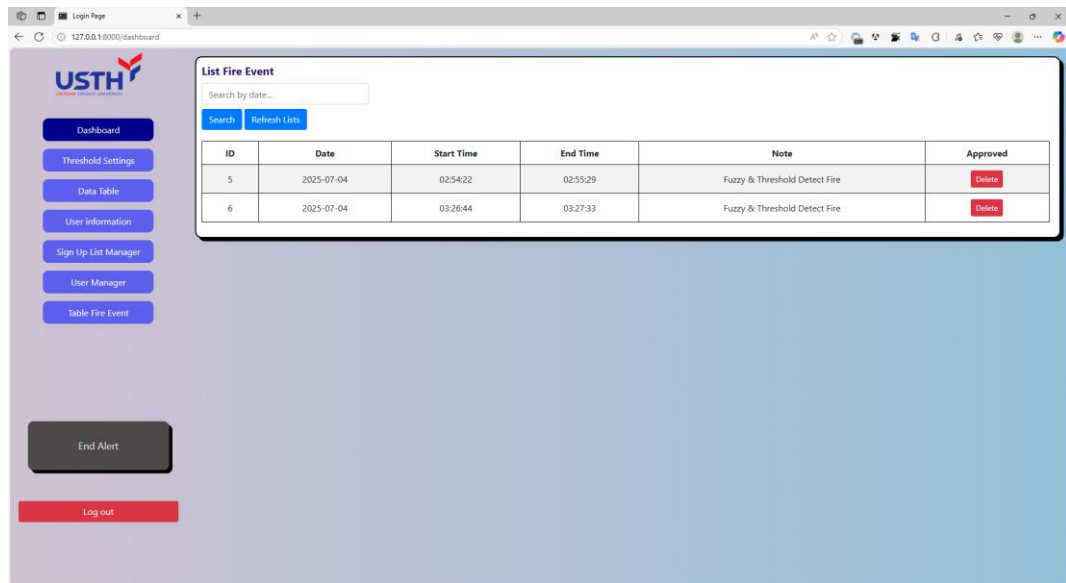


Figure 38: Software Interface – Fire Events.

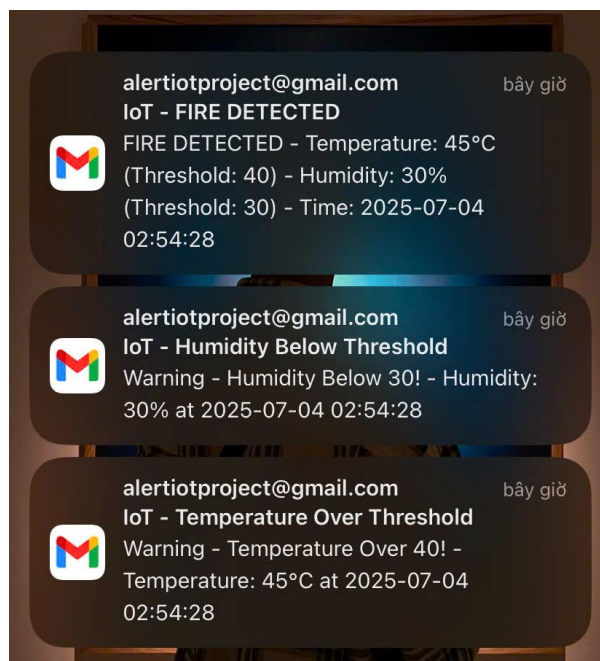


Figure 39: Software Interface – Warning Notification.

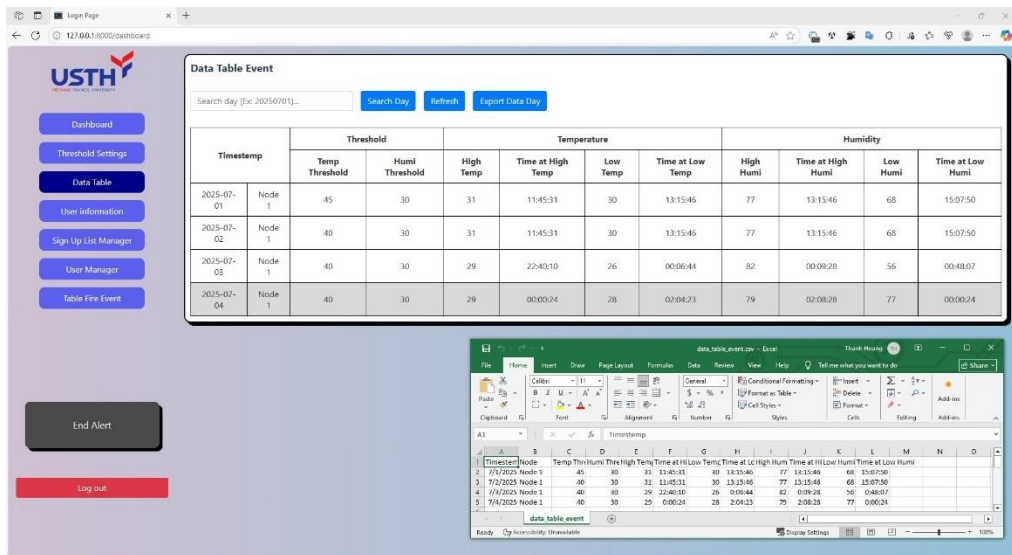


Figure 40: Software Interface – List and Export Data Day.

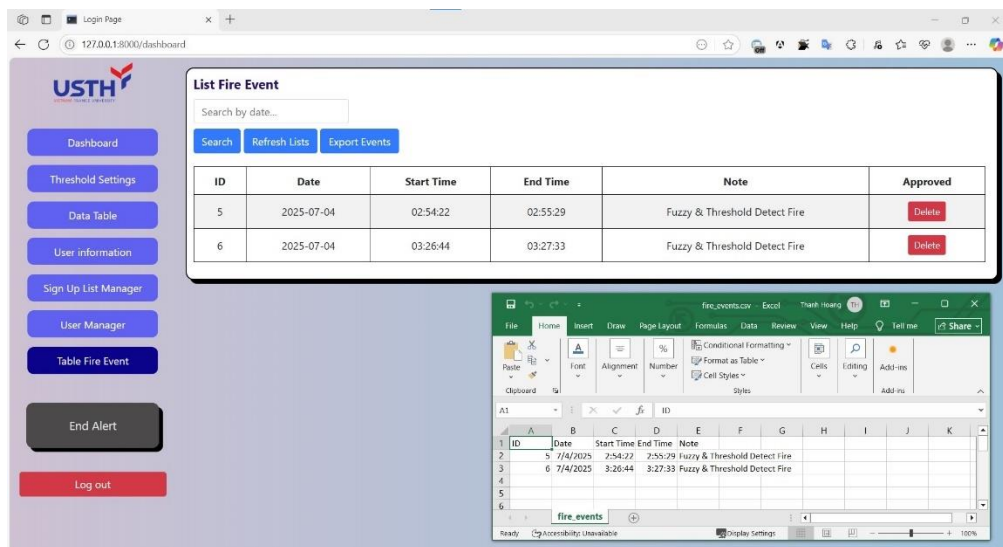


Figure 41: Software Interface – List and Export Fire Event.



Figure 42: The IoT-based monitoring and alert system.

Finally, as shown in Figure 42, the system has been fully implemented, covering both hardware and software. All features in Section 2.2 *Software & Firmware System Architecture* have been successfully deployed, enabling smooth interaction with users through a touchscreen monitor. Also, system alerts from the system to the user are implemented and working promptly.

3.2. Reliability and Error Rates

For this section, *Reliability and Error Rates*, the focus is on evaluating the reliability of environmental data collection, specifically examining packet loss during wireless transmission and the integrity of messages received by the CPU for processing and user display. In addition, user interaction delays, such as response times from input to system feedback, are also analyzed in this section.

Firstly, Temperature and Humidity data were collected in real time using the DHT11 sensor with high consistency and almost no data loss. The sensor responded correctly to changes in the environment. The KY-026 flame sensor also worked reliably, detecting heat from fire sources without delay. All data was packaged and encoded properly, then sent wirelessly to the receiver with a very low packet loss rate, thanks to channel encoding that improved data recovery. The Central Processing Unit received and decoded the data successfully.

After decoding, temperature, humidity, and fire status are displayed in real-time on the web interface, with no missing or delayed readings. The integration of Fuzzy Logic with Threshold allows the system to generate two types of alerts, making it more effective and accurate. These results confirm the system's reliability, quick response, and stable data transmission. Additionally, because of using VPN (TailScale), remote users can also access the system smoothly, with almost zero delay.

3.3. Evaluations and Challenges

The system developed in this thesis adopted wireless communication using the HC-12 RF module, inspired by previous research, *Chapter 1 – Section 1.3 Related Works*. This approach enabled low-cost, low-power, infrastructure-free data transmission. Early tests with direct wiring revealed challenges in protecting the CPU; therefore, the thesis aims to protect the CPU by using the sensor node that works independently. The use of separate nodes and a custom PCB significantly enhanced system stability, enabling future scalability through multi-node communication. Additionally, the web interface facilitated smooth user interaction, threshold setting, and alert notifications. Data transmission from the node to the CPU was consistent, with minimal data loss.

Several challenges were encountered during the development process. On the firmware side, working with STM32 in C was particularly demanding, requiring the configuration of pins, an external crystal, clocks, timers, and peripheral settings, as well as testing and debugging. On the software side, designing an accurate alert mechanism needed a careful balance of threshold-based detection and Fuzzy Logic. Additionally, deploying the button-end alert fire is very difficult, and implementing event logging and reliable reset mechanisms presents further technical challenges. Even though they were difficult, these challenges were essential in developing the system from a concept to a practical implementation and setting the stage for future expansion.

3.4. Conclusion

In summary, the system has been successfully implemented, covering both hardware and software, while demonstrating reliable operation across all components. The sensor node accurately collects environmental data, encodes it, and transmits it wirelessly with minimal packet loss. Meanwhile, the central processing unit receives, decodes, and displays the data in real-time through a user-friendly interface. The system responds promptly to fire-related events, and all key features, including the system alert, which combines Fuzzy Logic and Threshold alert, perform their intended functions smoothly. These features include alert functions and remote access through VPN. While certain challenges and areas for improvement remain, these results demonstrate the system's effectiveness, reliability, and real-time responsiveness, increasing its potential for application in real-world scenarios like machine rooms.

Conclusions and Perspective

This thesis proposes and develops a real-time fire monitoring and alert system based on IoT, using a multi-sensor node network to expand coverage, lower data transmission latency, and improve deployment flexibility. The system employs wireless communication to reduce cabling and ensure low latency, while offering an intuitive web interface that enables users to monitor and receive remote alerts effectively.

The system has demonstrated strong performance in real-world environments, showing high reliability in data collection and transmission, immediate alert response, distributed monitoring capability, and scalability. Separating the CPU location from the sensor nodes placed in high-risk areas also enhances overall system safety.

Nevertheless, there are still significant parts for improvement. Future enhancements will focus on: Integrating additional sensors (gas, smoke, vibration, environmental); Optimizing wireless data encoding algorithms for high-interference environments; Replacing the current VPN with a dedicated server for better availability and stability; Applying AI for fire detection/prediction and incorporating 360-degree thermal cameras if deploy at challenging enviroment liek forest; Improving software performance with lower-level languages (C/C++) and reinforcing system security; and Increasing system resilience via data buffering, backup batteries, and flame-retardant enclosures; Finally, the website's security was enhanced to protect fire event information, date data, and user information.

In conclusion, the proposed system represents a promising step toward a smart, flexible, and practical fire monitoring solution, particularly in scenarios that demand high reliability and real-time responsiveness. Although it is not yet a complete or perfect solution, this foundation holds significant potential for future development and real-world applications.

References

- [1] "Smart Home," no. <https://www.thebestsmart.homes/2022/09/smart-home-iot-smart-home-iot-solution.html>.
- [2] Pradeep, Aneesh; Latifov, Akmaljon; Yodgorov, Ahborkhuja; Mahkamjonkhojizoda, Nurislombek, "Hazard Detection using custom ESP32 Microcontroller and LoRa," *IEEE*, no. 26 May 2023, 2023.
- [3] N. I. M. E. N. M. Z. a. K. S. S. K. M. N. N N Mahzan, "Design of an Arduino-based home fire alarm system with GSM module," *Journal of Physics: Conference Series*, vol. 1019, 2917.
- [4] N. Hoesen, "Rancang Bangun Alat Pendeteksi Kebocoran Gas," *Jurnal Esensi Infokom*, vol. 5, 2021.
- [5] H. Zhang, X. Zuo, W. Jiang, Q. Jiang and G. Yin, "Design of Smart Home Environment Monitoring System Based on STM32," *International Symposium on Computer Technology and Information Science (ISCTIS)*, 2023.
- [6] H. Y. ·. J. L. ·. L. L. ·. Z. J. ·. Y. L. ·. D. Zhou, "Development of Four Rotor Fire Extinguishing System for Synchronized Monitoring of Air and Ground for Fire Fighting," *ICIRA*, p. 267–278, 2019.
- [7] N. Komalapati, V. C. Yarra, L. A. Vyas, Kancharla and T. N. Shankar, "Smart Fire Detection and Surveillance System Using IOT," *IEEE*, 2021.
- [8] A. E. H. A. A. S. S. A. K. A. B. A. Y. I. C. Kuldoshbay Avazov, "Forest Fire Detection and Notification Method Based on AI and IoT Approaches," *Future Internet (ISSN: 1999 - 5903)*, vol. 15, no. 2, 2023.
- [9] L. WHITE and R. AJAX, "Improved Fire Detection and Alarm Systems.," 2025.
- [10] M. S. B. Bahrudin, R. A. Kassim and N. Buniyamin, "Development of Fire alarm

system using Raspberry Pi and Arduino Uno," *2013 International Conference on Electrical, Electronics and System Engineering (ICEESE)*, 2013.

- [11] "SENSOR KIT X40," [Online]. Available: <https://sensorkit.joy-it.net/en/sensors/ky-026>.
- [12] "ASAIR SENSOR," [Online]. Available: <https://asairsensors.com/product/dht11-sensor/>.
- [13] "ePanorama.net," [Online]. Available: <https://www.epanorama.net/blog/2022/06/18/introduction-to-the-stm32-blue-pill-stm32duino-and-other-stm32-boards/>.
- [14] "HC Information Technology Co., Ltd.," [Online]. Available: <https://www.hc01.com/downloads/HC-12%20english%20datasheets.pdf>.
- [15] "HSHOP," [Online]. Available: <https://hshop.vn/anten-433mhz-35dbi-sma-duc>.
- [16] "The Register," [Online]. Available: https://www.theregister.com/2023/09/28/raspberry_pi_5_revealed/?td=amp-keepreading.
- [17] M. T. L. W. T. T. H. T. L. D. M. J. S. J. T. Ted Hsu, Asus - ASUSTeK Computer Inc - , 2 April 1989. [Online]. Available: <https://en.wikipedia.org/wiki/Asus>.
- [18] "ASUS," [Online]. Available: <https://www.asus.com/displays-desktops/monitors/zenscreen/>.
- [19] L. Meier, "MAVLINK," 2009. [Online]. Available: https://mavlink.io/en/guide/mavlink_version.html.
- [20] "MAVLINK," [Online]. Available: <https://mavlink.io/en/messages/common.html>.
- [21] V. R. Joan Daemen, "Wikipedia," 1998. [Online]. Available: https://en.wikipedia.org/wiki/Advanced_Encryption_Standard#cite_note-keysize-1.

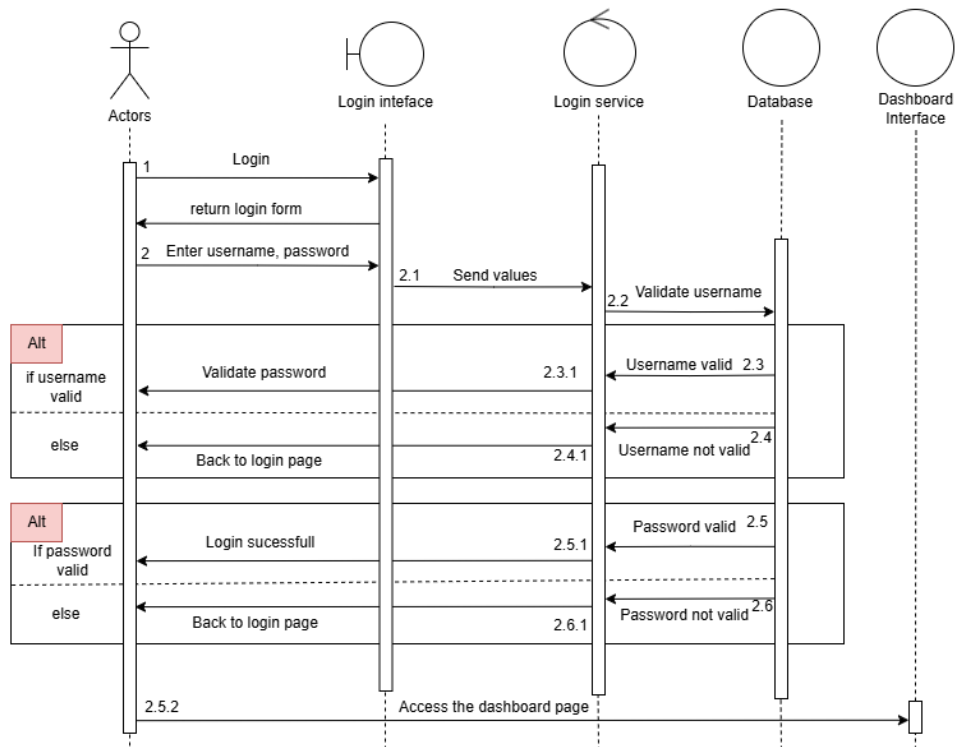
- [22] "Wikipedia," [Online]. Available: https://en.wikipedia.org/wiki/PKCS_7.
- [23] "Wikipedia," [Online]. Available: https://en.wikipedia.org/wiki/Linear-feedback_shift_register.
- [24] R. G. Gallager, "Low-Density Parity-Check Codes," *IRE Transactions on Information Theory*, vol. VIII, no. 1, pp. 21-28, 1962.
- [25] "Wikipedia," [Online]. Available: https://en.wikipedia.org/wiki/Fuzzy_logic.

Appendix

List of Figure Appendix:

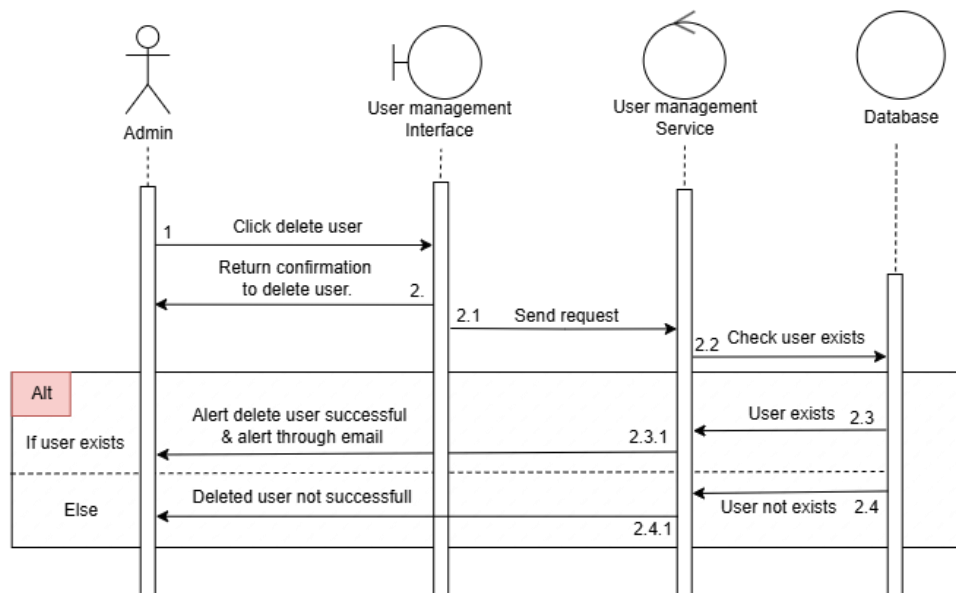
Appendix 1: Login Sequence Diagram.	47
Appendix 2: Delete User Sequence Diagram.	47
Appendix 3: Sign Up Sequence Diagram.....	48
Appendix 4: Create New User Sequence Diagram.	49
Appendix 5: Password Recovery Sequence Diagram.	49
Appendix 6: Edit User Information Diagram.	50
Appendix 7: Software Interface – Sign Up.	51
Appendix 8: Software Interface – Recovery Password.....	51
Appendix 9: Software Interface – Add New User.....	52
Appendix 10: Software Interface – List Sign Up User.	52
Appendix 11: Software Interface – List Account User.	53
Appendix 12: Software Interface – Account Information.	53

Login



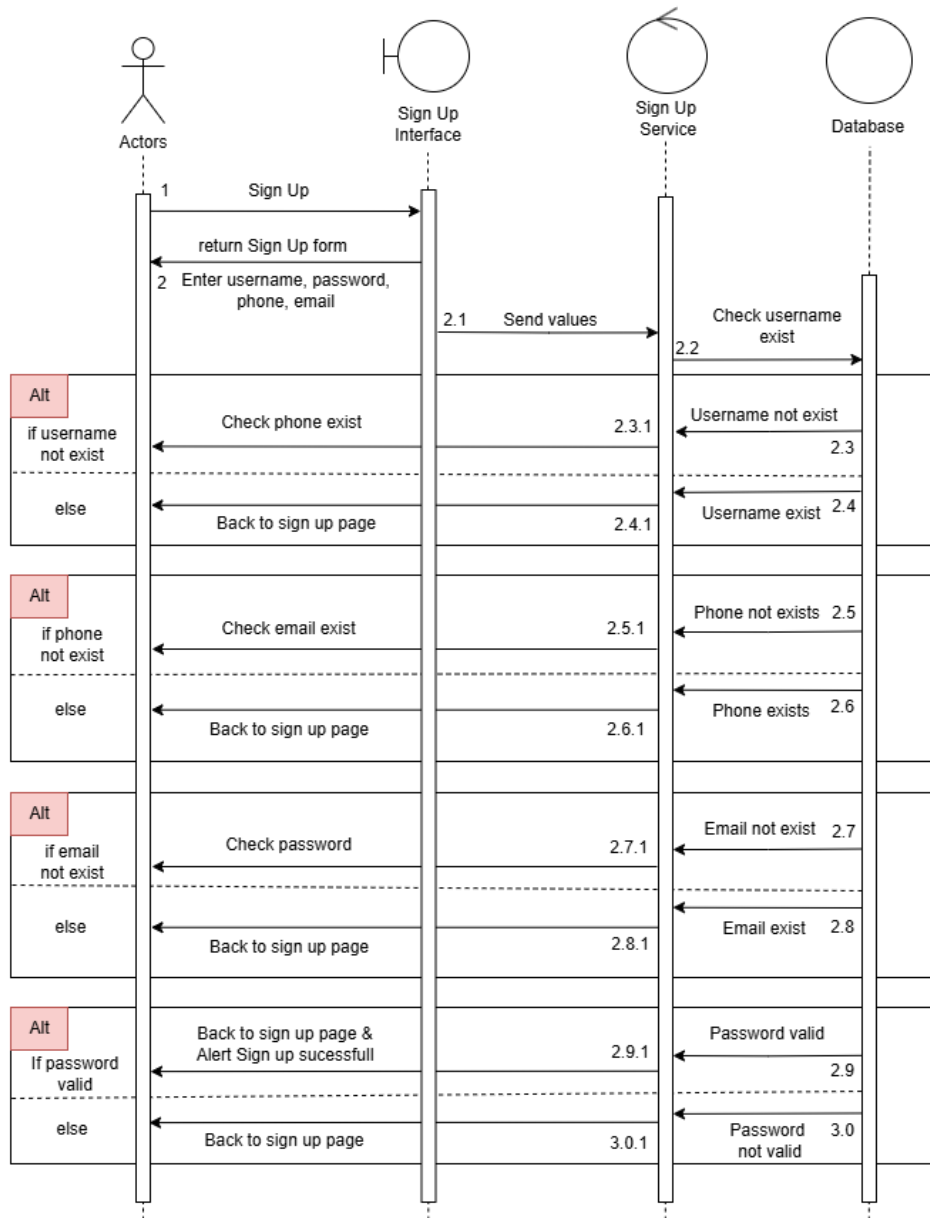
Appendix 1: Login Sequence Diagram.

Delete user



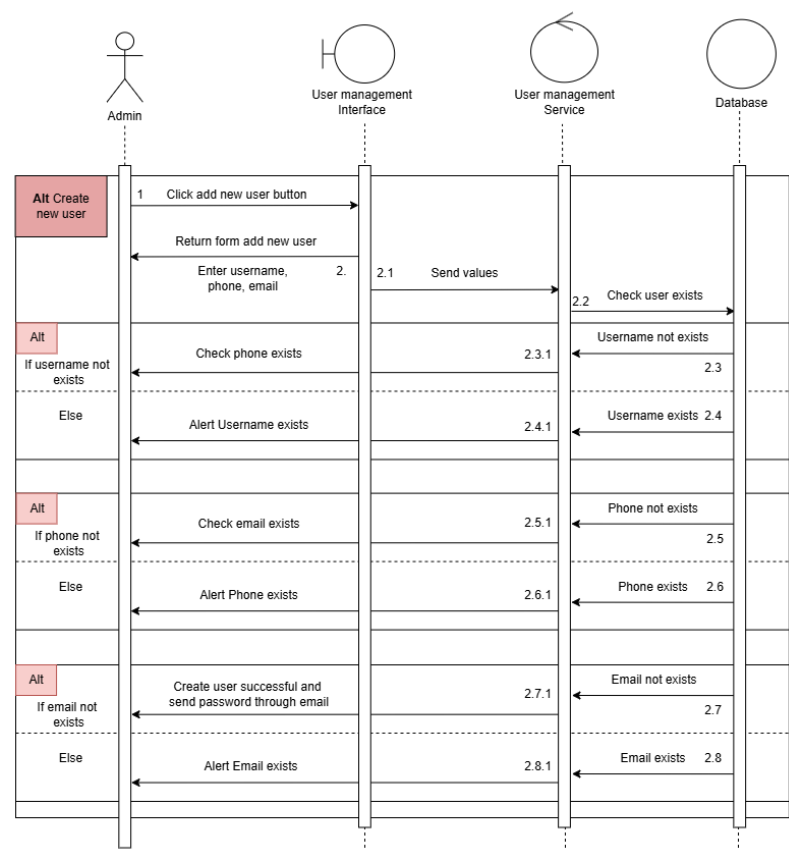
Appendix 2: Delete User Sequence Diagram.

Sign Up



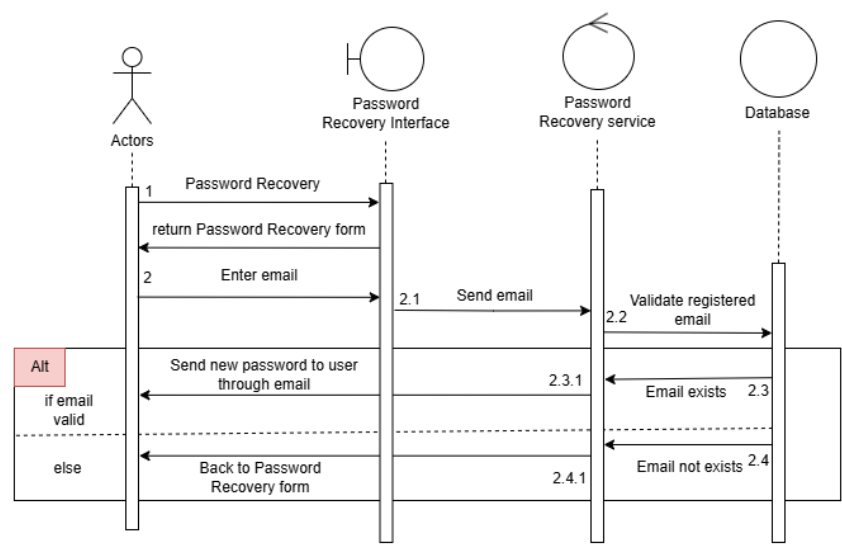
Appendix 3: Sign Up Sequence Diagram.

Create a new user



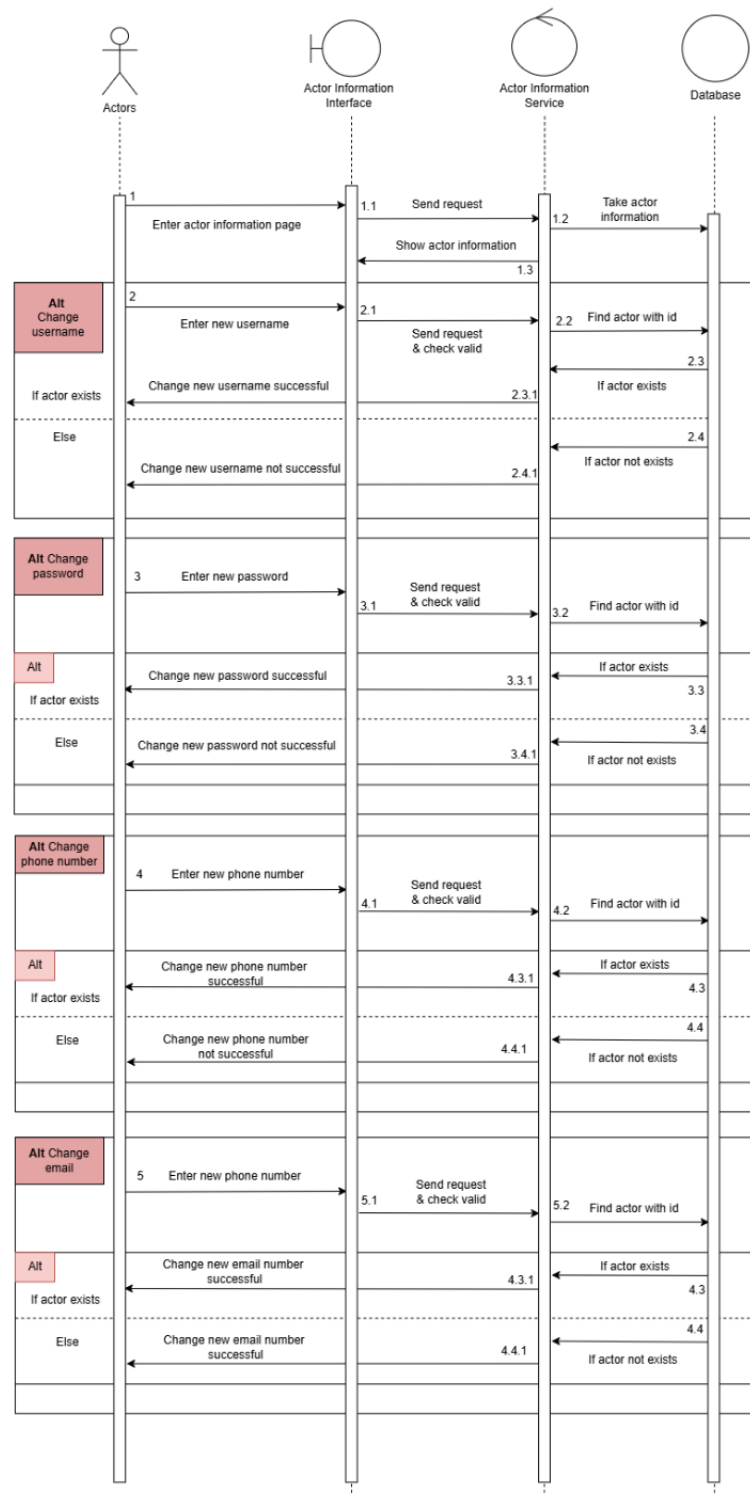
Appendix 4: Create New User Sequence Diagram.

Password Recovery



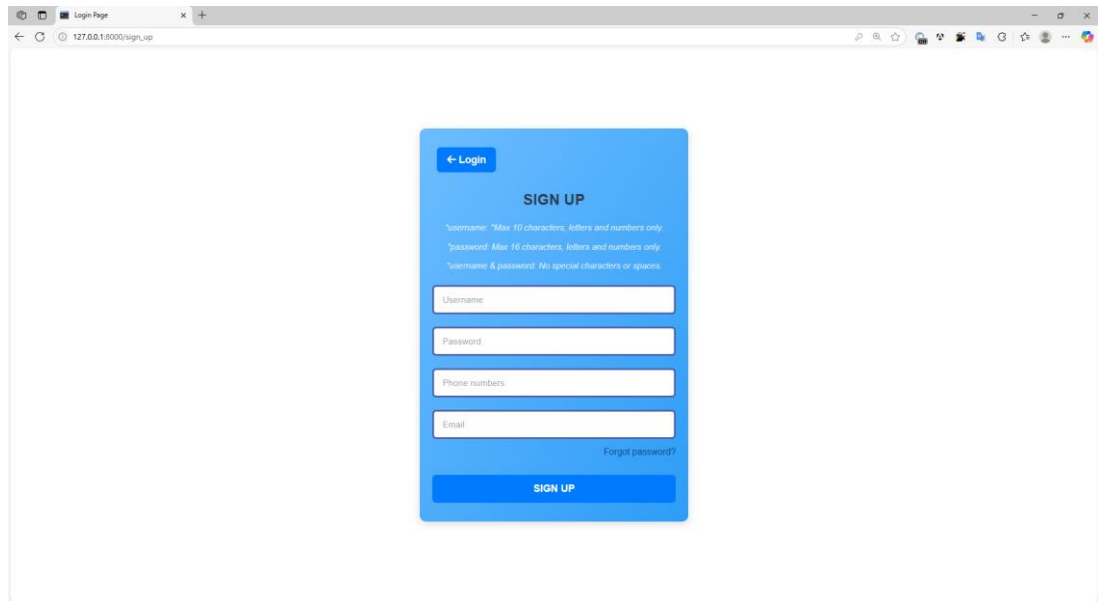
Appendix 5: Password Recovery Sequence Diagram.

Edit User Information



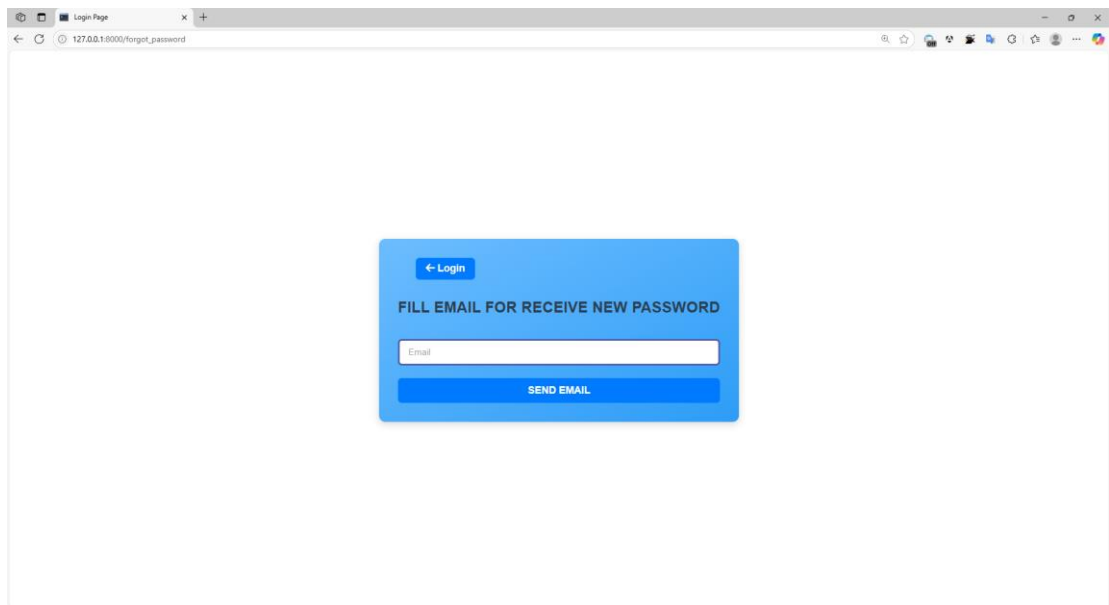
Appendix 6: Edit User Information Diagram.

Some pictures of the Software



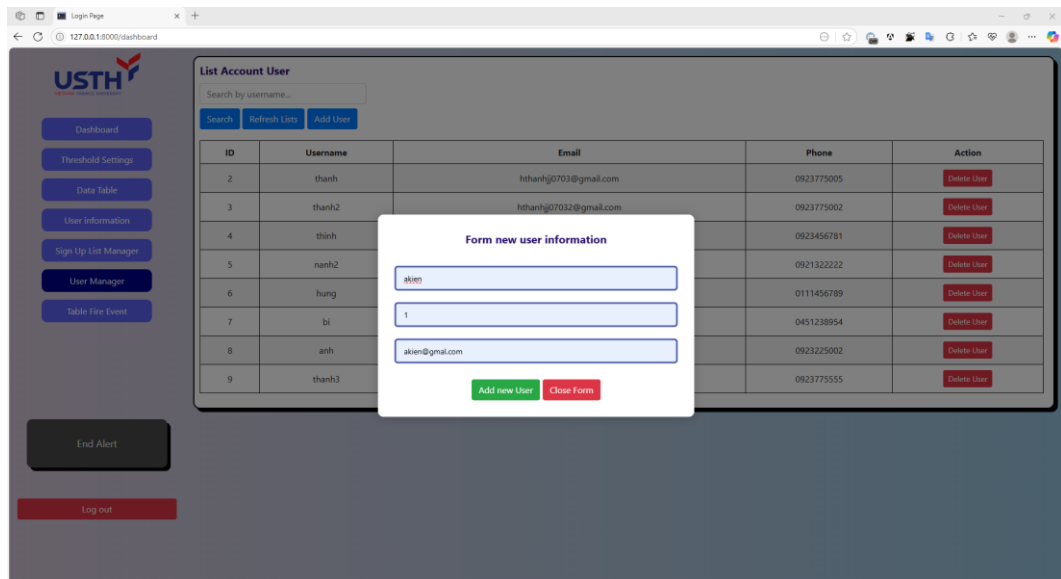
A screenshot of a web browser displaying a "Sign Up" form. The browser's address bar shows the URL "127.0.0.1:8000/signup". The form is centered on a white background and has a blue header with a "[← Login](\"#\")" link. The title "SIGN UP" is in bold. Below the title, there are three lines of small text: "Username: Max 10 characters, letters and numbers only", "Password: Max 16 characters, letters and numbers only", and "Username & password: No special characters or spaces". The form contains four input fields: "Username", "Password", "Phone numbers", and "Email". A blue "SIGN UP" button is at the bottom. A link "Forgot password?" is located to the right of the "Email" field.

Appendix 7: Software Interface – Sign Up.

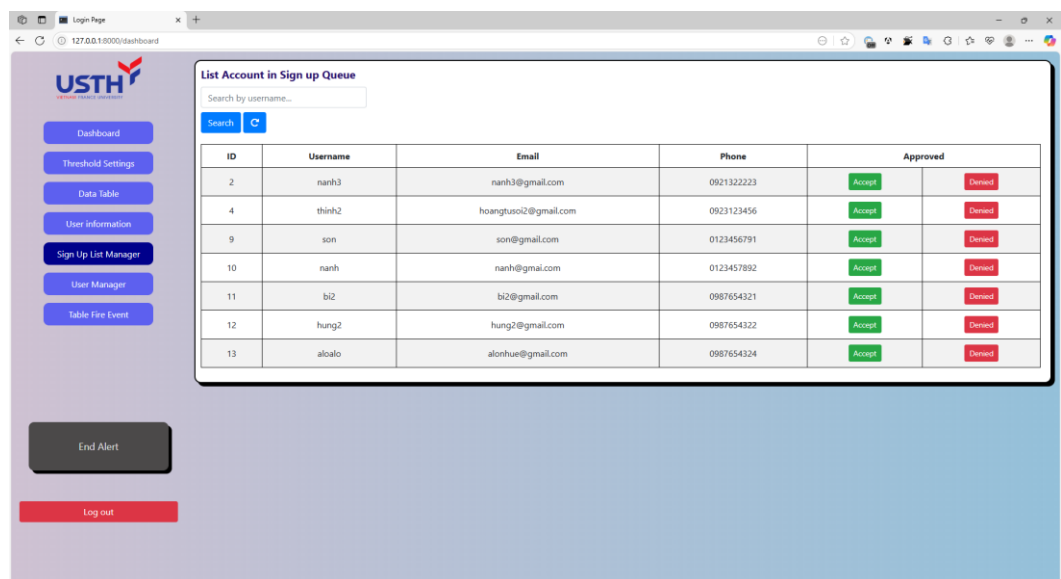


A screenshot of a web browser displaying a "Recovery Password" form. The browser's address bar shows the URL "127.0.0.1:8000/forgot_password". The form is centered on a white background and has a blue header with a "[← Login](\"#\")" link. The title "FILL EMAIL FOR RECEIVE NEW PASSWORD" is in bold. Below the title, there is a single input field labeled "Email". A blue "SEND EMAIL" button is at the bottom.

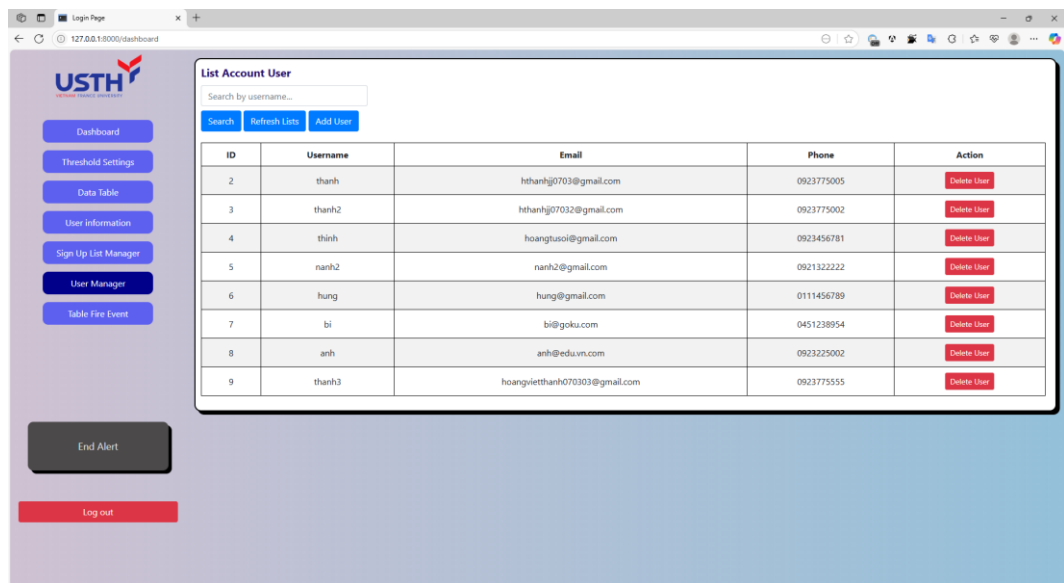
Appendix 8: Software Interface – Recovery Password.



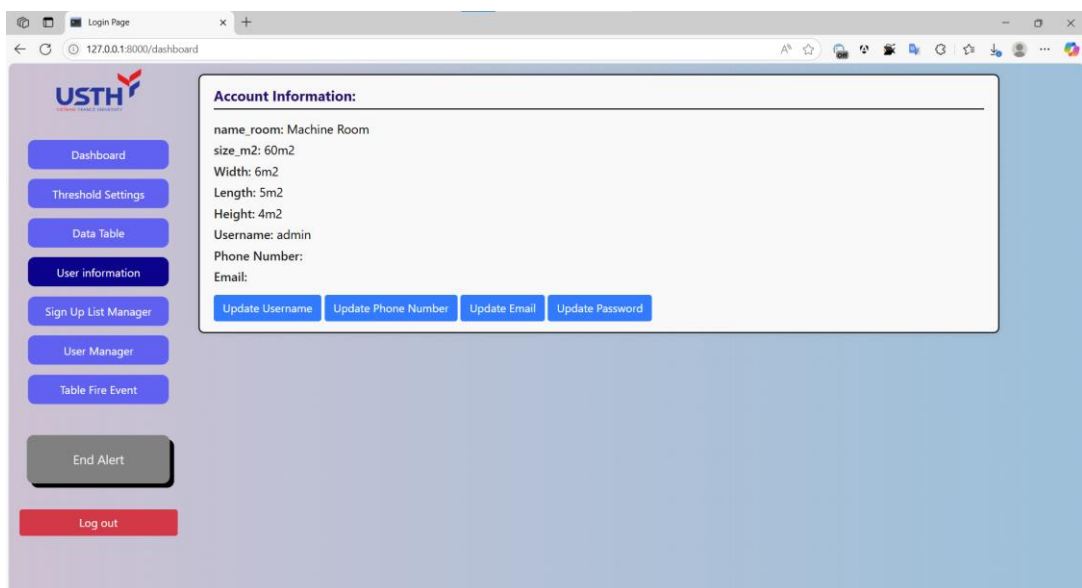
Appendix 9: Software Interface – Add New User.



Appendix 10: Software Interface – List Sign Up User.



Appendix 11: Software Interface – List Account User.



Appendix 12: Software Interface – Account Information.